Introducción

El diseño y la programación de un compilador se ha convertido, de una tarea artística, a un conjunto de metodologías sencillas para realizar las diferentes fases de un compilador.

En los libros de textos existentes que abarcan el tema de diseño de compiladores se mencionan técnicas específicas sobre análisis lexicográfico, análisis sintáctico, optimización de código etc. Pero, generalmente, lo que no se ha logrado estructurar de una manera clara es la fase de generación de código, tanto intermedio como objeto.

El principal propósito de la presente guía, es mostrar una técnica sencilla de generación de código, utilizando diagramas de sintaxis, de tal manera que el diseño de un traductor sea una labor completamente sencilla.

LENGUAJES

Lenguaje

Es un conjunto de vocablos, con los cuales se pueden estructurar ideas, de acuerdo a un patrón sintáctico.

TIPOS DE LENGUAJES

Lenguaje Natural

Lenguaje natural es el conjunto de vocablos por medio de los cuales el hombre elabora, expresa y comunica sus ideas.

Una característica fundamental del lenguaje natural es que el significado de una palabra puede depender del contexto en el que se ubique. Por ejemplo, analicemos las siguientes oraciones:

El niño *toma* leche
El niño *toma* el camión
El niño *toma* la mano
El niño *toma* el ejemplo

A pesar de que la secuencia sintáctica de las oraciones anteriores es la misma, el significado de la palabra *toma* depende de las palabras que le sucedan.

Lenguajes Artificiales

Un lenguaje artificial toma elementos propios y de otros lenguajes para formar su estructura. Por ejemplo, el Esperanto es un lenguaje artificial. Los lenguajes computacionales pertenecen a esta rama de lenguajes.

Lenguajes Artificiales Abstractos

Se le llama lenguaje abstracto a aquél que se define con el único propósito de experimentar nuevas estructuras o reglas de construcción.

Lenguajes Artificiales Computacionales

Se han llamado lenguajes artificiales computacionales a aquéllos que se emplean para expresar un algoritmo. Las estructuras que se forman son llamadas instrucciones y se emplean para programar una computadora. Un lenguaje artificial está formado por un léxico y está regido por una estructura sintáctica menos flexible, es decir, con menos alternativas, que la estructura sintáctica de un lenguaje natural.

Una característica de un lenguaje artificial es que el sinificado de una palabra siempre es el mismo, no depende del contexto en el que se encuentre.

En esta guía, concentraremos nuestra atención en el estudio de lenguajes artificiales y su procesamiento.

Los lenguajes artificiales computacionales, también conocidos como lenguajes de programación, están clasificados por niveles.

El **lenguaje de más bajo nivel** es el lenguaje maquinal. El hardware de la computadora sólo puede ejecutar las instrucciones expresadas en este lenguaje.

El **lenguaje ensamblador** permite el uso de mnemónicos para expresar instrucciones que pertenecen al lenguaje maquinal.

Los **lenguajes de alto nivel**, permiten el uso de palabras y de estructuras que hacen posible la conversión casi directa de un algoritmo en un programa.

Un lenguaje es de mayor nivel que otro, cuando su estructura sintáctica permite formar instrucciones que expresan un mayor número de instrucciones primitivas en lenguaje maquinal. Dado que una computadora sólo puede procesar instrucciones expresadas en lenguaje maquinal, se requiere de ***traductores*** para tener la posibilidad de ejecutar programas escritos en lenguajes de mayor nivel.

Estructura de un lenguaje

Anteriormente definimos lenguaje como un conjunto de vocablos con los cuales se pueden estructurar ideas. Ahora vamos a ampliar esta definición, de la siguiente manera:
Un lenguaje contiene un conjunto de vocablos que forman el **léxico** o vocabulario del lenguaje. En lenguajes naturales, a los vocablos se les llama simplemente palabras. Las palabras representan una imagen, ya sea física o abstracta. En lenguajes de programación, a los vocablos les llamaremos átomos o **tokens**.
Para definir el léxico de un lenguaje es posible emplear la notación de conjuntos, por ejemplo, el léxico del lenguaje español es: {árbol, casa, perro.....}. También se puede emplear una notación especial llamada expresiones regulares.
Las palabras y los tokens son concatenaciones de símbolos. Dichos símbolos forman parte de un **alfabeto**. Para definir el alfabeto de un lenguaje se utiliza notación de conjuntos. Ejemplo, el alfabeto del lenguaje español es: {a,b,c,d.....}

Para construir ideas, es necesario establecer una secuencia de palabras, respetando un patrón sintáctico. La **Sintaxis** tiene que ver con el orden en el que se acomoda el léxico; es el estudio de las relaciones permitidas entre las palabras. Un patrón sintáctico es aquél que muestra las normas rectoras de las relaciones permitidas entre los elementos del léxico de un lenguaje. Para definir las reglas sintácticas de un lenguajes existen varias alternativas: definir una gramática, un diagrama de sintaxis, o en algunos casos, una expresión regular.

Uns palabra guarda un significado, y éste depende de la imagen que representa, pero el significado puede variar dependiendo de las otras palabras con las que haya sido relacionada. Se le llama **semántica** al estudio del significado de las palabras. En los lenguajes de programación, la semántica de las instrucciones tiene una dependencia directa con la secuencia sintáctica que se haya utilizado, en cambio en los lenguajes naturales puede suceder lo que ya mencionamos anteriormente:

El niño toma el camión
El niño toma la mano

O peor aún, una misma oración puede representar varios significados, por ejemplo:

Te veo en el café - significado: te veré más tarde en una cafetería
Te veo en el café - significado: te veo reflejada en mi taza de café

Si pensamos en todas las complicaciones que involucra el análisis de un lenguaje natural, nos resultará sencillo pensar en el procesamiento de un lenguaje de programación.

Diseño de un Lenguaje de Programación

La tarea de definir un lenguaje involucra un análisis cuidadoso de la aplicación que tendrá, y del tipo de usuarios que lo van a utilizar. Cuando la persona que va a diseñar el lenguaje está muy familiarizado con diversos lenguajes de programación, es muy frecuente que caiga en la tentación de usar formatos que conoce, pero esto ocasiona que se olvide de los usuarios finales. Por ejemplo, si el lenguaje que se va a diseñar se integrará a un paquete administrativo, y va a tener un uso muy específico, y las personas que lo utilizarán no tienen una cultura computacional, entonces no tiene caso que las palabras reservadas estén escritas en inglés, y sería tedioso que los mensajes de error aparecieran sólo con una clave, o que hicieran mención a tecnisismos tales como "este procedimiento genera demasiado código", sería conveniente cambiar este mensaje por otro que dijera "favor de dividir su procedimiento, ya que contiene demasiadas instrucciones".

Los pasos que esta guía recomienda seguir para definir un lenguaje de programación son los siguientes:

- Buscar áreas de oportunidad

No se debe olvidar que al incluir un lenguaje a un sistema computacional, se generará la capacidad de resolver infinito número de problemas que pertenezcan a la misma clase. Una vez que definimos qué tipo de problemas queremos resolver, debemos experimentar con diferentes alternativas de planteamientos de solución de problemas involucrando en la experimentación a los posibles usuarios, y de ahí podremos observar los estatutos que son convenientes incluir en el lenguaje.

-Definir la orientación del lenguaje

Debe estar muy clara la orientación que se le va a dar al lenguaje ya que de ello depende el tipo de estatutos, el nivel o potencia de los estatutos y las estructuras de datos que se deberán incluir.

- Definir estatutos

Se recomienda no basarse completamente en algún otro lenguaje de programación ya que esto puede menguar la creatividad del diseñador. Lo que se puede hacer es tomar lo mejor de los lenguajes que existan y tengan la misma orientación y de ahí añadir innovaciones que den potencia al lenguaje y lo hagan práctico y útil.

No se deberá olvidar incluir estatutos de:

- **Entrada y salida.**
- **Condicionales.**
- **Ciclos.**
- **Asiganción.**
- **Referencias a subrutinas y/o métodos.**

-Definir estructuras de datos

Una de las labores más complejas de un traductor es la de resolver las estructuras de datos que permita el lenguaje, así es que se debe analizar detenidamente el tipo de estructuras que son estrictamente necesarias dada la orientación del lenguaje.

-Definir las expresiones

Las expresiones, tanto aritméticas como booleanas son parte primordial de un lenguaje, tomando en cuenta la orientación del lenguaje, es importante incluir operadores, que no necesariamente tienen que existir en la aritmética tradicional. Piense en el lenguaje C y observe lo útil que son operadores como el ++, *= etc. y que constituyeron una innovación introducida por este lenguaje. Una vez establecidos los operadores, se deberá fijar la jerarquía que seguirá la evaluación de una expresión.

-Definir las reglas semánticas
Se deberán definir reglas para las expresiones sin restringir demasiado la combinación de tipos dado que esto es fácil de manejar por el traductor, pero sin perder de vista la orientación del lenguaje. También se deberán definir reglas para los estatutos restringiendo sólo los casos en los que no se pueda hacer una traducción. Por ejemplo, en el estatuto CASE de Pascal no se puede utilizar un expresión que sea de tipo real o string, pero sería muy sencillo modificar el tipo de traducción que realiza el compilador para que estos tipos sean aceptados, de ahí podemos ver que es una restricción innecesaria.

- Definir la sintaxis
Para las palabras reservadas se deben de utilizar nombres mnemónicos, no muy largos, y de ser posible incluir sinónimos o abreviaciones, ya que para el compilador es sencillo contemplar esta posibilidad.
Debe ser congruente la secuencia de los estatutos, para que los usuarios puedan memorizar fácilmente la sintaxis del lenguaje, la terminación de los estatutos debe ser similar.

La definición de la sintaxis se debe construir sobre diagramas, ya que es la manera más clara de observar todos los órdenes posibles.
Los signos de puntuación se deben de incluir hasta el momento de definir los diagramas, y se deben incluir en caso de indeterminación o ambiguedad. En caso de que los signos sean necesarios, se deberán incluir a lo largo de todo el lenguaje. Si no son indispensables, entonces se puede incluir la posibilidad de que sean opcionales.
Si el método de análisis sintáctico que se va a seguir requiere de una gramática, entonces una vez definidos los diagramas de sintaxis, se podrá diseñar la gramática a partir de éstos. La primer gramática que se diseñe debe ser sencilla y que muestre una equivalencia clara con los diagramas. Si el método de análisis sintáctico requiere de que la gramática cumpla ciertas condiciones, entonces se deberá arreglar la gramática, conservando como parte de la documentación todas las versiones a las que se haya llegado, para ayudar a la depuración de errores.
Es conveniente conservar una copia de los diagramas de sintaxis con marcas que hagan referencia a los nombres de los símbolos no terminales que se utilizaron.

TEORÍA DE LENGUAJES

Es una notación utilizada para representar patrones de léxico y patrones sintácticos.

"STRINGS" Y LENGUAJES.

Un ALFABETO es un conjunto finito de símbolos, donde éstos son letras, dígitos o cualquier caracter. Por ejemplo el conjunto {0,1,2,3,4,5,6,7,8,9) es el alfabeto del sistema decimal.

Una CADENA o "STRING" es una secuencia finita de símbolos que son elementos de un alfabeto. Los números 1234 , 345, 987, 65 son algunos ejemplos de "strings" sobre el alfabeto decimal.

La LONGITUD de una cadena se representa por dos líneas verticales |23|, y da como resultado el número de símbolos de la cadena. Asi la cadena | compiladores | tiene longitud 12.

El "string" VACIO (o NULO) se representa por el símbolo ε y tiene una longitud cero.

Un LENGUAJE es cualquier conjunto de "strings" sobre un alfabeto.

OPERACIONES SOBRE STRINGS.

CONCATENACION.
La concatenación se representa por rs donde r y s son "strings" . Si r = libre y s = comercio el "string" resultante al realizar la concatenación de rs = librecomercio que es diferente de sr=comerciolibre.

El elemento identidad de la concatenación es el string vacío. Asi

$$r\varepsilon = \varepsilon r = r$$

EXPONENCIACION.

Para todo string S elevado al exponente i se obtiene

Si i = 0 entonces $S^0 = \varepsilon$

Si i > 0 entonces $S^i = S^{i-1} S$

OPERACIONES EN LENGUAJES

Existen muchas operaciones que se pueden aplicar a lenguajes. Para representar los patrones de léxico primero nos interesan la unión, concatenación y la cerradura (clousure), mismas que se definen en la siguiente tabla:

OPERACION	REPRESENTACION	DEFINICION
Union de L y M	L U M	L U M = { s I s está en L o M }
Concatenación de L y M	LM	LM = {m I I está en L y m está en M }
Kleene closure de L	L^*	$L^* = \bigcup_{i=0}^{\infty} L^i$
Positive closure de L	L^+	$L^+ = \bigcup_{i=1}^{\infty} L^i = L^* L$

EXPRESIONES REGULARES

• Las expresiones regulares son una notación especial que facilita la tarea de representar en forma precisa las reglas de léxico.

• Las expresiones regulares, al igual que las expresiones aritméticas, permiten que se realicen cierto tipo de operaciones sobre ellas.

• Las expresiones regulares son 'generadoras' de lenguajes, es decir, dada la expresión regular R, existe un lenguaje generado por R, denominado L(R).

TERMINOLOGIA UTILIZADA:

$\sum$	$\Rightarrow$	Alfabeto utilizado (conjunto finito de símbolos).
α	$\Rightarrow$	elemento del conjunto $\sum$
w,x,y	$\Rightarrow$	cadenas de símbolos tomados de $\sum$.
ε	$\Rightarrow$	cadena ("string") nula.
Φ	$\Rightarrow$	conjunto vacío.
$\sum^*$	$\Rightarrow$	conjunto de todas las cadenas generadas a partir de $\sum$. Contiene a ε como una cadena válida.
$\sum^+$	$\Rightarrow$	conjunto de todas las cadenas generadas a partir de $\sum$. En este conjunto ε es una cadena inválida.

OPERACIONES SOBRE UNA EXPRESION REGULAR.

Sean R y S dos expresiones regulares, entonces las operaciones que se pueden realizar sobre R y S se definen como:

1). Alternativa. Representa la unión de dos expresiones regulares, ésta se denota por:

$R \mid S$ ó $R + S$ ó $R \cup S$ $\qquad R \mid S = \{ w \mid (w \in R) \text{ ó } (w \in S) \}$

2). Concatenación. Representa la 'concatenación' de dos cadenas generadas por R y por S respectivamente, ésta se denota como:

$R \bullet S$ ó RS $\qquad RS = \{ wx \mid (w \in R) \text{ y } (x \in S) \}$

3). Exponenciación. Representa la repetición controlada de una expresión regular ('n' concatenaciones de la exp. regular consigo misma), esta operación se define recursivamente como:

$R^0 = \{\varepsilon\}$ $\qquad R^1 = R$ $\qquad R^i = R\,R^{i-1}$

4). Cerradura de Kleene (Kleene Clousure). Representa la repetición de una expresión regular (cero ó más concatenaciones de la exp. regular consigo misma), esta operación se define como:

$$R^* = \bigcup_{i=0}^{\infty} R^i = R^0 \cup R^1 \cup R^2 \ldots\ldots\ldots$$

5). Cerradura Positiva (Positive Clousure). Representa la repetición de una expresión regular, (una ó más concatenaciones de la exp. regular consigo misma) esta operación se define como:

$$R^+ = \bigcup_{i=1}^{\infty} R^i = R^1 \cup R^2 \ldots\ldots\ldots$$

6). Opción (selección). Permite que una expresión regular pueda ó no aparecer, ésta 'operación' se define como:

$$[\,R\,] \text{ ó } R^? = R^0 \cup R^1$$

• Formalmente, una expresión regular se define por:

Sea $\sum$ un alfabeto, entonces las expresiones regulares sobre $\sum$ y los conjuntos que denotan, se definen recursivamente como sigue:

1. Φ es una expresión regular que denota el conjunto vacío.
2. ε es una expresión regular y denota el conjunto $\{\varepsilon\}$, el cual es <> Φ
3. Para cada $\alpha \in \sum$, α es una expresión regular y denota el conjunto $\{\alpha\}$.
4. Si R y S son expresiones regulares que generan respectivamente L(R) y L(S), entonces:
   (R) | (S) es una expresión regular que representa a L(R) U L(S).
   (R) • (S) es una expresión regular que representa a L(R)•L(S)
   (R) es una expresión regular que representa a L(R)
   R^* es una expresión regular que representa a $(L(R))^*$
5. Las operaciones de Cerradura Positiva, Exponenciación y Selección se pueden representar a partir de las operaciones definidas en el punto anterior.
6. Ninguna expresión que no cumpla con los puntos arriba especificados será considerada una expresión regular.
   Los parentesis se pueden eliminar tomando en cuenta la siguiente jerarquía de operadores (se listan de mayor a menor jerarquía) y tienen asociatividad izquierda:

```
++  1. ( )
    2. cerraduras, exponenciación.
    3. concatenación.
--  4. alternativa (unión).
```

PROPIEDADES ALGEBRAICAS DE LAS EXPRESIONES REGULARES.

Las propiedades algebraicas son leyes que cumplen las expresiones regulares y ademas se utilizan en la manipulación de las mismas.

PROPIEDAD	DESCRIPCION
r | m = m | r	| es conmutativa
r | (m | g) = (r | m) | g	| es asociativa
(rm)g = r(mg)	la concatenación es asociativa
r(m|g) = rm | rg (m|g)r = mr | gr	la concatenación es distibutiva sobre la |
ε r = r r ε = r	ε es el elemento identidad de la concatenación
Φ | r = r |Φ = r	Φ es el elemento identidad de la alternativa
r* = (r | ε)*	la relación entre * y ε
r* = (r +| ε)	la relación entre *, + y ε
r** = r*	* es idempotente

TEORIA DE GRAMATICAS

GRAMATICA.

Notación formal empleada para representar las reglas sintácticas de un lenguaje.

Una gramática se compone de los siguientes tres elementos:

1. Un conjunto finito de símbolos **terminales (T).**
   donde un símbolo terminal es cada uno de los lexemas que genera Léxico.

2. Un conjunto finito de símbolos llamados **no_terminales (N)** que incluye al símbolo inicial **S** (**Símbolo sentencial**).
   Un símbolo No_terminal es una variable útil para agrupar cadenas de símbolos gramaticales (terminales y no_terminales/variables sintácticas) que forman construcciones sintácticas.

3. Un conjunto finito de **producciones (P)** de la forma:

   String1 → String2

   donde :
   - String1 puede ser cualquier secuencia de símbolos gramaticales que tiene al menos una variable sintáctica (no_terminal).
   - String2 es cualquier secuencia de símbolos gramaticales.

Se utilizan para especificar la manera en la cual los símbolos gramaticales se pueden combinar para formar patrones sintácticos válidos.

Sin embargo la forma más general de representar una gramática es mediante un cuadrúplo de la forma: G = (**N** , **T** , **P** , **S**).

Ejemplo de una gramática es: G_1 = ({X, S}, {a, b}, P, S) donde:
a y b son terminales, X y S son no terminales, S es el símbolo sentencial, y las producciones (P) son:

$$S \rightarrow XS$$
$$S \rightarrow \varepsilon$$
$$X \rightarrow aX$$
$$X \rightarrow a$$
$$aaaX \rightarrow ba$$

DERIVACION.

Una **derivación** en una gramática es una serie de terminales y no_terminales que se obtienen, por medio de sustituciones utilizando las producciones de la gramática, donde la primera inicia en el símbolo sentencial S, y terminan cuando tenemos un string formado solamente de terminales.

Ejemplo. derivar el "string" ababa utilizando la gramática G_1 anterior

$$S \Rightarrow XS \Rightarrow aXS \Rightarrow aaXS \Rightarrow aaaXS \Rightarrow aaaaXS \Rightarrow abaXS \Rightarrow abaaXS \Rightarrow abaaaXS \Rightarrow abaaaaXS \Rightarrow ababaS \Rightarrow ababa$$

LENGUAJE
El **lenguaje** generado por una gramática es el conjunto de todos los strings formados solamente de símbolos terminales que pueden ser derivados a partir del símbolo sentencial S.

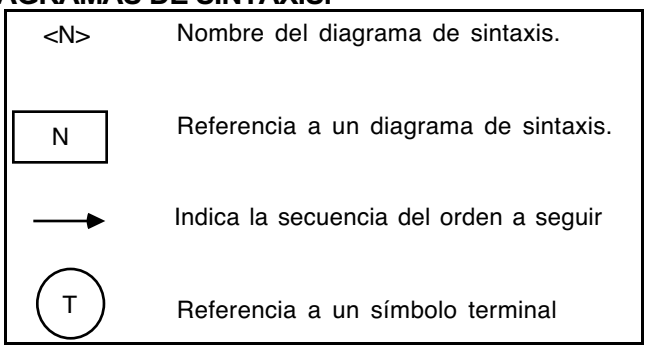
ARBOL SINTACTICO ó ARBOL DE DERIVACION.
Un **árbol sintáctico** es la representación gráfica de una derivación.
Las hojas del árbol son terminales y los nodos son variables sintácticas que, leídas de izquierda a derecha (en cualquiera de los niveles del árbol) crean lo que se conoce como forma sentencial.

DIAGRAMAS DE SINTAXIS / GRAMÁTICAS GRÁFICAS

Otra alternativa para una gramática es representarla en forma gráfica utilizando Diagramas de Sintaxis.

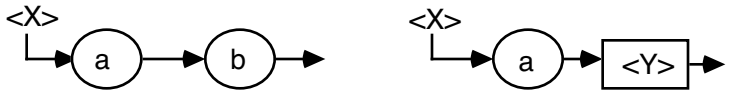
En términos generales, un diagrama de sintaxis se define como una herramienta útil para representar, de manera sencilla, el orden que deben seguir los elementos del lenguaje. La aplicación principal de los mismos, como su nombre lo indica, es representar los patrones sintácticos de los lenguajes, utilizando para ello uno o varios diagramas, dentro de los cuales debe existir uno que lleve el nombre del símbolo sentencial de la gramática.
A continuación se presenta la notación utilizada para definir los diagramas de sintaxis.

NOTACION DE LOS DIAGRAMAS DE SINTAXIS.



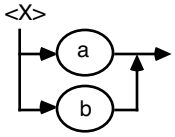
Los diagrama de sintaxis poseen patrones que se repiten constantemente por lo tanto es de suma importancia definirlos, ya que ello facilitará el diseño de los mismos. A continuación se presenta un conjunto de diagramas de sintaxis con los patrones sintácticos (o estructuras sintácticas) más comunes :

a) Secuencia de símbolos



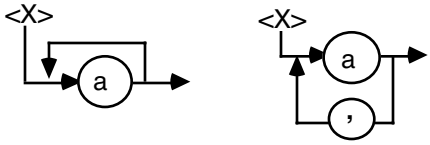
El primer diagrama indica que el patrón sintáctico <X> está formada por 'a' seguido de 'b'.
El segundo diagrama indica que <X> inicia con el símbolo 'a' seguido por el patrón sintáctico <Y>.

b) Alternativa entre varios símbolos



Este diagrama indica que la estructura <X> puede ser formada indistintamente por el símbolo 'a' ó por el símbolo 'b' pero no ambos.

c) Estructuras Cíclicas



El primer diagrama indica que <X> aceptará cadenas formadas por una ó más 'a'; mientras que el segundo aceptará más de una 'a' siempre y cuando venga separada por una ','.

Cuando una estructura sintáctica es muy compleja, es recomendable fraccionarla en pequeños diagramas que representen operaciones básicas.

Clasificación de Gramáticas

Debido al tipo de representación que se utiliza para denotar las reglas sintácticas de un lenguaje, las gramáticas se dividen en dos grandes grupos:

Gramáticas Gráficas.

Comúnmente llamadas Diagramas de Sintaxis. Sirven para representar en forma esquemática ciertos tipos de reglas sintácticas. No son tan generales como las gramáticas formales. Los expertos No las consideran gramáticas formales, debido a la falta de definición matemática de éstas y a que no aceptan los mismos tipos de lenguajes que las gramáticas formales.

Gramáticas Formales.

Son gramáticas que se utilizan para representar las reglas de construcción de diversos tipos de lenguajes. Estas gramáticas forman parte de la "Teoría de los lenguajes formales". Uno de los principales investigadores en esta área es Noam Chomsky, quien en 1959 publicó ***"Propiedades Formales de las Gramáticas"*** en el cual mostró el modelo matemático correspondiente a una gramática para la representación del lenguaje natural. Adicionalmente proporcionó una clasificación general para las gramáticas, partiendo de las características particulares de los lenguajes que cada una de ellas acepta. Actualmente esa clasificación es ampliamente aceptada y se le conoce como : **"JERARQUIA DE CHOMSKY".** En esta jerarquía se distinguen 4 tipos principales de gramáticas, los cuales son:

0.- Sin Restricciones (Unrestricted grammars UG ó Tipo 0)
Como su nombre lo indica, las reglas de producción de estas gramáticas no siguen ningún patrón prestablecido, no existe ningún tipo de restricción para su construcción. No son útiles para representar los lenguajes artificiales (de programación).

1.- Sensitivas al contexto (Context_Sensitive grammars CSG ó Tipo 1)
Son gramáticas cuyas reglas de producción presentan ciertas restricciones para su construcción. En estas gramáticas, las reglas de producción siempre se presentan como:
$X \rightarrow Y$
en las que, $|X| <= |Y|$, donde $|X|$ representa la longitud de la cadena de símbolos gramaticales.
Adicionalmente, X y Y son elementos de $(N \cup T)^*$ y X contiene al menos un elemento de N y Y no puede llegar a ser vacío. Se llaman Sensitivas al contexto porque las sustituciones que se

realizan de los símbolos No-terminales (por su forma sentencial ó lado derecho de la regla de producción correspondiente) se llevan a cabo únicamente si ese símbolo aparece en el contexto adecuado. Los lenguajes que se generan a partir de una gramática sensitiva al contexto se denominan Lenguajes Sensitivos al Contexto.

2.- Libre de contexto (Context_Free grammars CFG ó Tipo 2)
En este tipo de gramáticas, las reglas de producción presentan restricciones adicionales a las que se tenían el las CSG. En este tipo de gramáticas, las reglas de producción son de la forma:
$X \rightarrow Y$
donde, X es elemento de los símbolos No-terminales y $| X | = 1$, es decir, sólo existe un símbolo del lado izquierdo de la producción y es No-terminal. Además Y es una cadena de símbolos gramaticales que son elementos de $(N \cup T)^*$. En estas gramáticas, la producción $X \rightarrow \varepsilon$ (que denota a la cadena nula) es válida. Aunque siempre existirá una gramática equivalente que no utilice este tipo de producciones.

Se llaman libre de contexto porque las sustituciones que se realizan de los símbolos No-terminales (por su forma sentencial ó lado derecho de la regla de producción correspondiente) se llevan a cabo independientemente del contexto en que aparezcan (es decir, no importando qué símbolos se encuentren en la vecindad del símbolo a sustituir). Aún cuando las gramáticas libres de contexto no son lo suficientemente poderosas para representar el lenguaje natural, éstas son las más útiles para representar los lenguajes artificiales. Los lenguajes que se generan a partir de una gramática libre de contexto se denominan Lenguajes libres de contexto.

3.- Regulares (Regular grammars RG ó Tipo 3)
Las reglas de producción de este tipo de gramáticas presentan la mayor cantidad de restricciones para su construcción. Dichas reglas de producción son de la forma: $X \rightarrow Y$
donde, X es elemento de los símbolos No-terminales y $| X | = 1$, es decir, sólo existe un símbolo del lado izquierdo de la producción y es No-terminal. Además Y es una cadena de símbolos gramaticales con elementos de $(N \cup T)^*$, pero $| Y | <= 2$.
Si $| Y | = 1$, el símbolo que forma a Y deberá ser un elemento de los Terminales; por el contrario si $| Y | = 2$, Y tendrá la forma aB ó Ba, donde a es elemento de los Terminales y B será elemento de los No-terminales. En estas gramáticas, la producción $X \rightarrow \varepsilon$ (que denota a la cadena nula) es válida si y sólo si X es el símbolo inicial de la gramática.
Dentro de este tipo de gramáticas, existe una subdivisión adicional que depende de la forma que tengan las reglas de producción, dicha subdivisión se define como:

- **Gramáticas Lineales Derechas :**
  Aquí todas las producciones son de la forma: $X \rightarrow a$ ó $X \rightarrow aB$.
- **Gramáticas Lineales Izquierdas :**
  Aquí todas las producciones son de la forma: $X \rightarrow a$ ó $X \rightarrow Ba$.

Los lenguajes que se generan a partir de gramáticas regulares son denominados Lenguajes regulares, para los cuales también existe una representación en un Autómata de Estados Finitos y en una expresión regular.

La jerarquía de estas gramáticas y las relaciones entre ellas, se da como:

$L(G3) \subset L(G2) \subset L(G1) \subset L(G0)$ donde $\subset$ significa subconjunto de

En la siguiente tabla se muestra un resuen de los tipos de gramáticas y las restricciones que tienen las producciones de cada una de ellas.

JERARQUIA DE CHOMSKY(1959).

tipo	Nombre	Restricciones en las producciones $X \rightarrow Y$
0	Sin Restricciones (Unrestricted Grammar)	X = cualquier string, al menos tiene un símbolo no_terminal Y = cualquier string
1	Sensitivas al Contexto (Context Sensitive Grammar)	X = cualquier string, al menos tiene un símbolo no_terminal Y = cualquier string de longitud igual o mayor que la de X.
2	Libres de Contexto (Context Free Grammar)	X = un solo símbolo no_terminal Y = cualquier string
3	Regulares (Regular Grammar)	X = un solo símbolo no_terminal Y = TN o Y = T Y = NT o Y = T

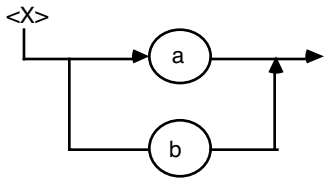
Ahora que se conoce la clasificación de las gramáticas formales, cabe señalar que las gramáticas gráficas (diagramas de sintaxis) son útiles para representar únicamente gramáticas libres de contexto y gramáticas regulares. Esa es una de las razones por las que no son reconocidas estrictamente como gramáticas.

EQUIVALENCIAS ENTRE DIAGRAMAS Y GRAMATICAS LIBRES DE CONTEXTO.

A continuación se presenta un conjunto de diagramas de sintaxis con los patrones sintácticos más comunes, junto con la gramática libre de contexto y la expresion regular equivalentes.

PATRONES SINTACTICOS MAS COMUNES.

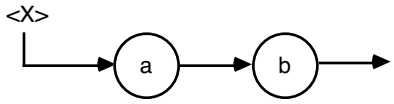
a) Estructura de alternativa.



Gramática equivalente
X->a
X->b

Expresión Regular
X=(alb)

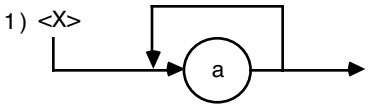
b) Estructura de Secuencia.



Gramática equivalente
X->ab

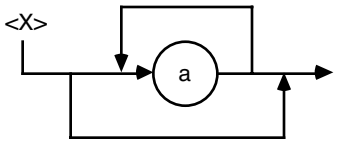
Expresión Regular
X=ab

c) Estructuras cíclicas.

1) 

Gramáticas equivalentes
1) X->Xa
X->a
2) X->aX
X->a

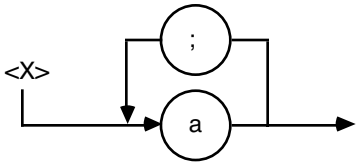
Expresión Regular
$+X = a$

2) 

Gramáticas equivalentes
1) X->Xa
X->ε
2) X->aX
X->ε

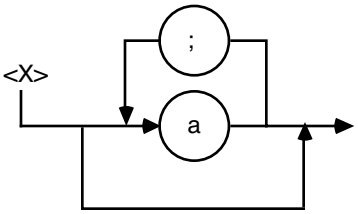
Expresión Regular
$X = a^*$

d) Estructuras cíclicas con separador.

1) 

Gramáticas equivalentes
1) X->X;a
X->a
2) X->a;X
X->a

Expresión Regular
X=a(;a)*

2) 

Gramática equivalente
X->aY
X->ε
Y->;aY
Y->ε

Expresión Regular
X= (ε | a(;a)*)

TRANSFORMACION DE UN DIAGRAMA A UNA GRAMATICA LIBRE DE CONTEXTO.

1. Marcar en el diagrama de sintaxis los patrones sintácticos que existan.

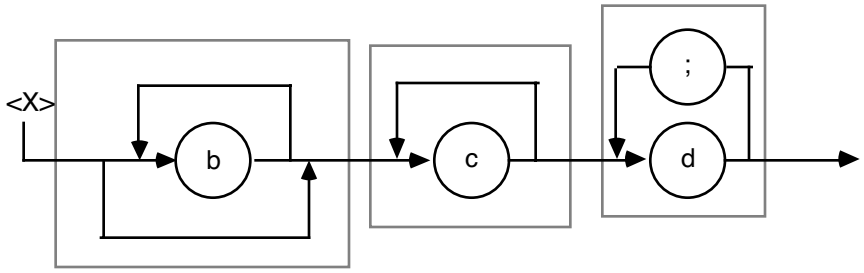
2. Asignar un símbolo no terminal que no exista a cada uno de los patrones que se marcaron en el paso anterior.

3. Obtener la gramática libre de contexto equivalente de cada uno de los patrones.

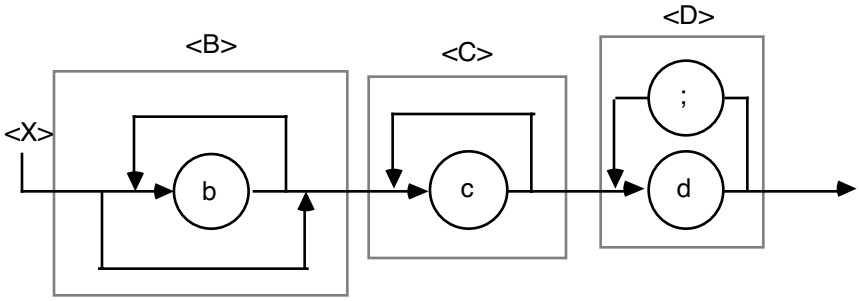
4. Obtener la(s) produccion(es) del diagrama utilizando los símbolo no terminales que identifican a cada patron.

A continuación se muestran los cuatro pasos a seguir para obtener la gramática libre de contexto equivalente al diagrama de sintaxis siguiente:

1.



2.



3. <B> → ε <C> → c <D> → d
   <B> → b <B> <C> → c <C> D> → d ; <D>

4. <X> → <B><C><D>

Definición General de una Gramática (sin importar su clasificación).
Una gramática se representa en un cuádruplo de forma:

G = (N , T , P , S)

donde:

N = Conjunto de Símbolos No-Terminales (o variables sintácticas).

T = Conjunto de Símbolos Terminales (lexemas). Siempre se cumple que: $N \cap T = \Phi$

S = Símbolo No-Terminal que se distingue por generar todas las cadenas válidas para un lenguaje definido. Se le denomina Símbolo Inicial ó Símbolo Sentencial;

donde ($S \in N$, pero $S \notin (N \cup T)^*$)

NOTA: $(N \cup T)^*$ Es el conjunto de símbolos gramaticales. $(N \cup T)^*$ representa todas las posibles combinaciones.

P = Conjunto no vacío de relaciones que van de $(N \cup T)^* N (N \cup T)^*$ hacia $(N \cup T)^*$, en general : $P \in (N \cup T)^* \times (N \cup T)^*$

Lo anterior significa que P es un subconjunto de todas las posibles relaciones entre los símbolos gramáticales.

La representación de esta relación es:

$\alpha \rightarrow \beta$ donde $\alpha \in (N \cup T)^* N (N \cup T)^*$ y $\beta \in (N \cup T)^*$

α es llamado el lado izquierdo y **β** el lado derecho de la relación.

Estas relaciones son llamadas reglas sintácticas o de sustitución, también se le llaman producciones (ya que son el resultado de un producto).

DERIVACIONES ó SUSTITUCIONES

La sustitución o derivación, formalmente se define como, una relación binaria ($\Rightarrow$) sobre el conjunto $(N \cup T)^*$, tal que, para cualquier $_1$ y $_2 \in (N \cup T)^*$ y cualquier producción $\alpha \rightarrow \beta$ se define $\omega_1 \alpha \omega_2 \Rightarrow \omega_1 \beta \omega_2$, donde ω_1 y ω_2 pueden ser strings nulos (ε).

Sea **G= (N,T,P,S)** una gramática; entonces para cualquier ψ , $\sigma \in (N \cup T)^*$ se dice que σ es DIRECTAMENTE DERIVABLE de ψ ($\psi \Rightarrow \sigma$) si existen strings ω_1 y $\omega_2 \in (N \cup T)^*$ tales que

$\psi = \omega_1 \alpha \omega_2$ y $\sigma = \omega_1 \beta \omega_2$ y $\alpha \rightarrow \beta \in P$

También se denomina ψ produce directamente a σ ó σ se reduce directamente a ψ .
Es el resultado de aplicar UNA SOLA regla.

Sea **G= (N,T,P,S)** una gramática; entonces para cualquier ψ , $\sigma \in (N \cup T)^*$ se dice que ψ produce a ó deriva a σ ($\psi \overset{+}{\Rightarrow} \sigma$) si existen strings ω_0 , $\omega_1 \ldots, \omega_n$ (n>0) $\in (N \cup T)^*$ tales que

$\psi = \omega_0 \Rightarrow \omega_1 \Rightarrow \ldots\ldots \Rightarrow \omega_n = \sigma$ (n pasos de derivación)

La relación $\overset{+}{\Rightarrow}$ es la cerradura transitiva de $\Rightarrow$. Si n = 0, entonces se define la cerradura transitiva reflexiva de ™ como:

$\psi \overset{*}{\Rightarrow} \sigma \Leftrightarrow \psi \overset{+}{\Rightarrow} \sigma$ ó $\psi = \sigma$

FORMA SENTENCIAL

Sea **G= (N,T,P,S)** una gramática. Sea $\sigma \in (N \cup T)^*$. Se dice que σ es una Forma Sentencial de G $\Leftrightarrow$ $S \overset{*}{\Rightarrow} \sigma$ (derivando a partir del Símbolo Inicial se obtiene σ)

SENTENCIA (ORACION)

Se dice que σ es una Sentencia de G $\Leftrightarrow S \overset{*}{\Rightarrow} \sigma$ y $\sigma \in T^*$ (Toda sentencia es una Forma Sentencial)

ARBOL DE DERIVACION (ó Sintáctico).

Sea G= (N,T,P,S) una gramática. Se dice que un árbol T es un Arbol de Derivación para G si:

1. Cada nodo del árbol tiene una etiqueta X (su nombre), donde $X \in (N \cup T)^*$.
2. El nombre de la raíz es S (símbolo inicial).
3. Si un nodo N tiene al menos un descendiente, y dicho nodo se llama X, entonces $X \in N$
4. Si los nodos N_1, N_2,...N_m son descendientes directos de N, en orden de izquierda a derecha y se llaman respectivamente A_1, A_2,....,A_m, entonces
$$A \rightarrow A_1A_2....A_m \quad \in \quad P$$
5. Si un nodo N se llama ε, entonces es un nodo hoja y es el único descendiente de su padre.
6. Ninguna otra cosa puede ser considerado un árbol de derivación.

Todas las hojas de un árbol de derivación de izquierda a derecha representan una forma sentencial para G.

TIPOS DE DERIVACION

El orden en que se llevan a cabo las sustituciones determina el tipo de derivación que se está utilizando. Las derivaciones pueden ser : Más a la Izquierda (M.I.) , Más a la derecha (M.D.) o Aleatorio. (La más utilizada de las tres es la derivación Más Izquierda)

DERIVACION DE MAS A LA IZQUIERDA.

En cada paso de derivación SIEMPRE se sustituye al símbolo No-Terminal que se encuentra más a la izquierda en la Forma Sentencial. Este tipo de derivación se representa como :

$$S \overset{+}{\underset{MI}{\Rightarrow}} \sigma$$

DERIVACION DE MAS A LA DERECHA.

En cada paso de derivación SIEMPRE se sustituye al símbolo No-Terminal que se encuentra más a la derecha en la Forma Sentencial. Este tipo de derivación se representa como :

$$S \overset{+}{\underset{MD}{\Rightarrow}} \sigma$$

DERIVACION ALEATORIA.

No existe un orden para realizar las sustituciones. (No tiene aplicación práctica).

Ejemplo: G = ({S,L}, {i, , , (,) } , P, S }

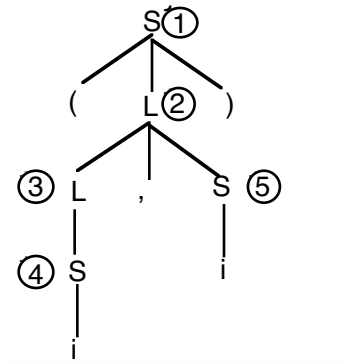
1) $S \rightarrow (L)$ 2) $S \rightarrow i$ 3) $L \rightarrow L , S$ 4) $L \rightarrow S$

La derivación de más a la izquierda de la expresión (i, i) es:

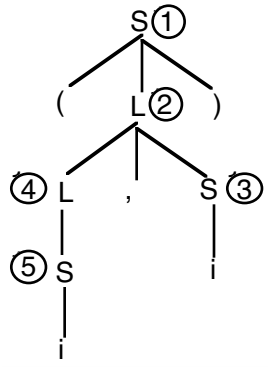
$S \rightarrow (L) \rightarrow (L , S) \rightarrow (S , S) \rightarrow (i , S) \rightarrow (i , i)$

La derivación de más a la derecha de la expresión (i, i) es:

$S \rightarrow (L) \rightarrow (L , S) \rightarrow (L , i) \rightarrow (S , i) \rightarrow (i , i)$



Arbol de derivación de más a la izquierda



Arbol de derivación de más a la derecha

Los númeron indican el orden en que los símbolos no terminales fueron derivados.

LENGUAJE GENERADO POR UNA GRAMATICA G. L(G)
Sea G= (N,T,P,S) una gramática. El lenguaje generado por G, llamado L(G) es el conjunto de todos los strings formados por SIMBOLOS TERMINALES, tales que:

$$L(G) = \{ w / (w \in T^*) \text{ y } (S \overset{*}{\Rightarrow} w) \}$$

Ambigüedad en Gramáticas

A continuación se presentan conceptos importantes dentro del estudio de las características de las gramáticas:

AMBIGÜEDAD:
Sea G = { N , T , P , S } una gramática libre de contexto y sea L(G) el lenguaje generado por esa gramática.
Si existe un string **w** (donde **w** $\in$ L(G)) para el cual existen dos ó más formas de realizar la derivación de más a la izquierda ($\mathbf{S} \overset{+}{\underset{MI}{\Rightarrow}} \mathbf{w}$) ó existen dos ó más formas de realizar la derivación de más a la derecha ($\mathbf{S} \overset{+}{\underset{MD}{\Rightarrow}} \mathbf{w}$), entonces se dice que: G es una gramática ambigua.

TIPOS DE AMBIGÜEDAD:
Dentro del estudio de gramáticas existen dos tipos fundamentales de ambigüedad, los cuales son:

Ambigüedad Inherente:
Las gramáticas que presentan este tipo de ambigüedad no pueden utilizarse para lenguajes de programación, ya que por más transformaciónes que se realicen sobre ellas, NUNCA se podrá eliminar completamente la ambigüedad que presentan.

Ambigüedad Transitoria:
Este tipo de ambigüedad puede llegar a ser eliminada realizando una serie de transformaciones sobre la gramática original. Una vez que se logra lo anterior, la gramática queda lista para ser reconocida por la mayor parte de los analizadores sintácticos. (Se le considera "ambigüedad" porque existen métodos para realizar análisis sintáctico que no aceptan gramáticas con estas características)

Dónde se presenta la Ambigüedad Transitoria:
Generalmente la ambigüedad se presenta cuando existen producciones con factores comunes (distintas alternativas para un símbolo no-terminal que inician de la misma forma); ó cuando existen producciones que son recursivas izquierdas (producciones para un símbolo no-terminal en las cuales el primer símbolo de su forma sentencial es ese mismo símbolo no-terminal).

¿Cómo solucionar el problema de la Ambigüedad Transitoria?:
Para eliminar este tipo de ambigüedad, es necesario, primero eliminar:
- Factores comunes izquierdos inmediatos y No-inmediatos.
- Recursividad izquierda inmediata y No-inmediata.

OPERACIONES SOBRE GRAMATICAS LIBRES DE CONTEXTO: ELIMINACIÓN DE AMBIGÜEDAD TRANSITORIA

FACTORIZACION DE TERMINOS COMUNES IZQUIERDOS INMEDIATOS.

Existen gramáticas que tiene producciones de la forma

$A \rightarrow \underline{\alpha}\,\beta_1 \mid \underline{\alpha}\,\beta_2$

como por ejemplo

$S \rightarrow \underline{i\ E\ t\ S}\ e\ S \mid \underline{i\ E\ t\ S}$

donde $\underline{\alpha}$ es el término común en las producciones de A.

Sin embargo para poder llevar a cabo el análisis sintáctico de las mismas mediante algunas técnicas se debe eliminar los términos comunes izquierdos llevando a cabo el proceso de factorización siguiente:

Las producciones $A \rightarrow \underline{\alpha}\,\beta_1 \mid \underline{\alpha}\,\beta_2$ se transforman en las siguientes

$A \rightarrow \underline{\alpha}\ A'$
$A' \rightarrow \beta \mid \beta_2$

las cuales nos generan el mismo lenguaje. Existe un nuevo símbolo no terminal A´ en la gramática, el cual no altera la gramática del lenguaje.

Generalizando el procedimiento para n producciones de A que tienen factor común izquierdo:

1. Agrupar todas las producciones de A, sin importar cuantas sean.
   $A \rightarrow \alpha\,\beta_1 \mid \alpha\,\beta_2 \mid \ldots \mid \alpha\,\beta_n \mid \lambda$
   *donde λ representa otras producciones de A que no tienen factor común izquierdo .
2. Remplazar las producciones de A a un conjunto equivalente mediante la siguiente transformación
   $A \rightarrow \alpha\ A' \mid \lambda$
   $A' \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

ELIMINACION DE RECURSIVIDAD IZQUIERDA INMEDIATA.

Una gramática tiene recursividad izquierda si tiene un no terminal A tal que existe una derivación $A \Rightarrow A\,\alpha$ para algún string α. Algunas técnicas de análisis sintáctico no pueden manejar gramáticas con recursividad izquierda por ello se debe eliminar primero.

Pasos para eliminar la recursividad izquierda inmediata.

1. Agrupar todas las producciones de A, sin importar cuantas sean.

   $A \rightarrow A\,\alpha_1 \mid A\,\alpha_2 \mid \ldots \mid A\,\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

   donde β_i no inicia con A.

2. Reemplazar las producciones de A a un conjunto equivalente mediante la siguiente transformación :

   $A \rightarrow \beta_1\,A' \mid \beta_2\,A' \mid \ldots \mid \beta_n\,A'$

   $A' \rightarrow \alpha_1\,A' \mid \alpha_2\,A' \mid \ldots \mid \alpha_m\,A' \mid \varepsilon$

En el caso de obtener producciones con recursividad izquierda al momento de realizar este segundo paso, se debe repetir el procedimiento.

ELIMINACION DE LA AMBIGÜEDAD TRANSITORIA NO-INMEDIATA:

Este tipo de ambigüedad se presenta cuando, después de realizar un conjunto de sustituciones se generan factores comunes ó recursividad izquierda. Para poder eliminarla, ee deben sustituir todas las alternativas de los símbolos No-terminales involucrados para convertir esa ambigüedad No-inmediata en inmediata, para posteriormente aplicar las reglas expuestas anteriormente.

OPERACIONES SOBRE GRAMATICAS LIBRES DE CONTEXTO: CÁLCULO DE FIRST's Y FOLLOW's

CALCULO DE FIRST PARA UNA GRAMATICA.

El **FIRST(α)**, donde α es cualquier secuencia de símbolos $(N \cup T)^*$, será un conjunto de símbolos terminales, con los que pueden iniciar las derivaciónes a partir de α.

Para calcular el **FIRST(α)** para todos los símbolos de α, aplicar las siguientes reglas hasta que no se puedan añadir más terminales ó ε a cualquier **FIRST**.

1. Si X es terminal, entonces añadir X al FIRST(X).
2. Si $X \rightarrow \varepsilon$ es una producción de G, entonces añadir ε al FIRST(X).
3. Si $X \rightarrow Y_1 Y_2 Y_3 Y_4 \ldots Y_K$ es una producción de G entonces

   -añadir el FIRST(Y_1) al FIRST(X).

   -añadir el FIRST(Y_2) al FIRST(X) si y solo si el FIRST(Y_1) tiene ε.

   : : : :

   -añadir el FIRST(Y_k) al FIRST(X) si y solo si el FIRST(Y_1) & FIRST(Y_2) .. &FIRST(Y_{k-1}) tienen ε.

   -y por último añadir ε al FIRST(X) si y solo el sí el FIRST(Y_1) & FIRST(Y_2) .. & FIRST(Y_k) tienen ε.

CALCULO DE FOLLOW PARA UNA GRAMATICA.

El FOLLOW(A), para un no terminal A, será el conjunto de terminales a que pueden aparecer inmediatamente a la derecha del no terminal A en alguna forma sentencial, esto es, el conjunto de terminales a que pueden existir en una derivación de la forma $S \Rightarrow \alpha A a \beta$ para alguna α y β. Si A es el símbolo no terminal más a la derecha en alguna forma sentencial, entonces $ (eof) está en el FOLLOW(A).

Para calcular el FOLLOW(A) para todos los no_terminales A de la gramática G, aplicar las siguientes reglas a cada producción hasta que ya no se puedan añadir elementos al conjunto de FOLLOW.

1. Añadir el $ en el FOLLOW(S), donde S es el símbolo sentencial de la gramática y $ es el símbolo que marca el fin de la entrada.
2. Si existe una producción $A \rightarrow \alpha B \beta$ en la gramática G, entonces todo lo que esté en el FIRST(β) se añade en el FOLLOW(B), excepto ε.
3. Si existe una producción $A \rightarrow \alpha B$ o una producción $A \rightarrow \alpha B \beta$ en la gramática G, donde el FIRST(β) contiene ε, entonces añadir todo lo que este en el FOLLOW(A) en el FOLLOW(B).

ANÁLISIS DE SINTAXIS

Metodologías TOP-DOWN

Es aquel análisis sintáctico que inicia la derivación de un string a partir del símbolo sentencial y trata de encontrar la derivación más a la izquierda para el "string" que se está analizando. Es un tipo de metodología EXPANSIVA, ya que, partiendo únicamente del Símbolo Sentencial, va expandiendo el árbol hasta obtener la secuencia de tokens más parecida al string de entrada que sí es válida para el lenguaje.

Las metodologías Top-Down tienen la ventaja de ser "muy sencillas", sin embargo, dada la naturaleza de su construcción aceptan un número limitado de gramáticas (no soportan ambigüedades). Entre las más comunes se encuentran:

- Descenso Recursivo (también llamado Predictivo Recursivo).
- El método Predictivo (también llamado Predictivo NO-Recursivo).

Metodologías BOTTOM-UP

Conocido tambien como **shift-reduce parsing,** construye un árbol sintáctico para un string de entrada iniciando en las hojas del árbol (bottom) y lleva a cabo reemplazamientos hasta llegar a la raíz (símbolo sentencial). En general reduce un string **w** al símbolo sentencial de la gramática. Es un tipo de metodología denominada REDUCCIONISTA ya que, partiendo de las hojas de un "supuesto" árbol de derivación, va realizando reducciones hasta llegar a la raíz del árbol. Si el archivo de entrada es correcto y, en cada paso se eligió la sustitución adecuadamente, se obtendrá un Derivación más a la Derecha en Reversa.

Estas metodologías no son tan evidentes como las Top-Down, sin embargo son mucho más poderosas que éstas y soportan una mayor cantidad de gramáticas (incluyendo algunas ambigüedades temporales). Entre las más comunes se encuentran:

- La familia de metodologías LR: LR-Simple, LR-Canónico y LALR.

Ejemplo. Para G = (a, b, c, d, e | A, B, S | S)

1) S → aABe
2) A → Abc
3) A → b
4) B → d

El análisis bottom-up del string abbcde es:

```
          3          2          4          1
abbcde → aAbcde → aAde → aABe → S
```

Los números indican el número de la producción que se aplicó.

El análisis Top-down del string abbcde es:

```
 1          2          3          4
S → aABe → aAbcBe → abbcBe → abbcde
```

MÉTODOS TOP-DOWN

Estas metodologías tratan de encontrar el árbol con la derivación de más a la izquierda para un string de entrada. Son técnicas EXPANSIVAS.

DESCENSO RECURSIVO (PREDICTIVO RECURSIVO)

Es la técnica más sencilla que existe para realizar el análisis sintáctico, sin embargo requiere demasiada programación (genera muchas líneas de código).
Además es el único método que permite realizar el análisis sintáctico a partir de la definición directa de los diagramas de sintaxis que representan a un lenguaje; ya que todas las demás requieren la gramática formal.

Esta técnica consiste en :

- Implementar una rutina (método ó función) para cada símbolo No-Terminal (var. sintáctica) que se tenga en los diagramas de sintaxis. Esta rutina debe considerar todas las posibles variantes (caminos) definidas para esa variable sintáctica en particular.
- Programar un estatuto condicional (IF-THEN_ELSE) para cada uno de los símbolos que aparezcan en los diagramas.

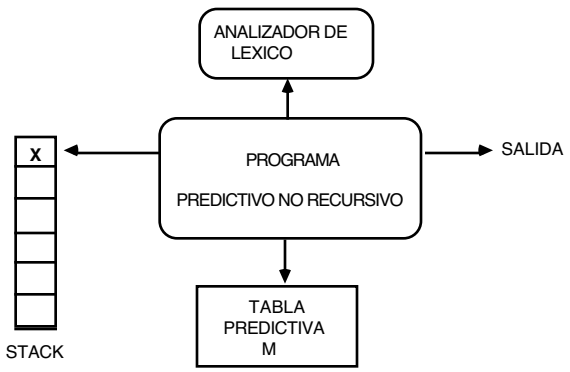
El análisis comienza en el diagrama principal del lenguaje y va solicitando tokens al léxico conforme *'acepta'* el token que actualmente analiza. Si el token que envía léxico no era el esperado por la sintaxis se generará un error del tipo *"Esperaba : _____ "*

MÉTODO PREDICTIVO (PREDICTIVO NO-RECURSIVO)

Para poder llevar a cabo el análisis sintáctico utilizando la técnica predictiva no recursiva es necesario que se haya eliminado la recursividad izquierda y los términos comunes izquierdos de la gramática del lenguaje que será analizado sintacticamente (Esta ambigüedad no es soportada por las metodologías Top-Down).

MODELO DE UN ANALIZADOR PREDICTIVO NO RECURSIVO.

Es posible tener un "parser" predictivo teniendo un "stack" explícitamente, para simular el proceso de las llamadas recursivas. El parser decide la producción que se utilizará en la derivación sólo en base al "token" actual y al símbolo no terminal **X** que estó en el top del "stack" en ese instante.



- El STACK tendrá una secuencia de símbolos de la gramática y un **$** en el fondo del "stack".
- La TABLA PREDICTIVA es un arreglo de dos dimensiones M[**X**,**a**], donde **X** es un símbolo no terminal, y **a** es símbolo terminal o el símbolo **$** que es enviado por el léxico indicando fin de archivo.

Algoritmo de Manejo de la Matriz PREDICTIVA (DRIVER) :

Entrada : Un archivo a analizar
Salida : Mensaje con : Entrada Aceptada ó Entrada Errónea.
Proceso:
Inicio
Repetir lo siguiente:
Sea X el símbolo que está en el tope de la pila y Nexttoken = el símbolo de entrada actual.
Si X es elemento del Conjunto de Terminales entonces:
Si X = Nexttoken = $ entonces:
Acepta el string de entrada
Si no entonces:
Si X = Nexttoken y <> $ entonces:
Sacar a X de la pila y remover a Nexttoken de la entrada *(*es un símbolo válido*)*
Si no entonces:
ERROR *(*Esperaba X *)*
Si no entonces : (* X es elemento de los No-Terminales *)
Si M [X, nexttoken] = $X \rightarrow Y_1\ Y_2\ Y_3\ .....\ Y_n$ entonces:
Sacar a X de la pila.
Meter a la pila $Y_n\ ....\ Y_3\ Y_2\ Y_1$ *(* quedando Y1 en el TOP de la pila *)*
Si no entonces: *(* casilla vacía *)*
ERROR *(* Se esperaba alguno de los First de X *)*
Hasta que Nexttoken = $ y la Pila esté vacía.
Fin

Algoritmo de Construcción de la Matriz PREDICTIVA

Entrada : Una gramática G.
Salida : La matriz predictiva correspondiente a dicha gramática.
Proceso :
Inicio
Para cada producción $A \rightarrow \alpha$ de la gramática G hacer
Para cada símbolo terminal a en los FIRST(α) hacer
Añadir $A \rightarrow \alpha$ en M[A,a].
Si existe el símbolo ε dentro de los FIRST(α)
Para cada terminal b (b <> $) en el conjunto de FOLLOW(A) hacer
Añadir $A \rightarrow \alpha$ en M[A,b]
Si ε está en los FIRST(α) y el símbolo $ está en los FOLLOW(A)
Añadir $A \rightarrow \alpha$ en M[A,$].
Para cada casilla que quedó vacía en la matriz hacer
ERROR *(* Símbolo inválido para esa producción *)*
Fin.

Estas metodologías tratan de encontrar el árbol con la derivación de más a la derecha en reversa para un string de entrada. Son técnicas REDUCCIONISTAS.

Algoritmo para el manejo de la MATRIZ generada por cualquiera de los MÉTODOS LR

FUNCIONAMIENTO DEL ANALIZADOR L.R. :

1. Suposiciones:
   - La variable Nexttoken se usará para almacenar el símbolo de la entrada que se está analizando.
   - La variable TAcción se utilizará para almacenar la Tabla de Acciones.
   - La variable TGoto se utilizará para almacenar la Tabla de Brincos.
   - Existe un OBSERVA que regresa lo que está almacenado en el tope de la pila sin sacarlo.

2. Inicialización:
   - Se tiene un string de entrada con el símbolo $ (fin de entrada concatenado al final).
   - Nexttoken := Primer símbolo de la entrada.
   - Hacer que la pila esté vacía.
   - Insertar en la Pila un 0 { Inicialmente el analizador está en el estado 0 }
   - TAcción := contenido de la tabla de acciones.
   - TGoto := contenido de la tabla de brincos.

3. Algoritmo :

- REPETIR, HASTA QUE SE ACEPTE O MARQUE ERROR, LO SIGUIENTE:
  - Nexttoken contiene el elemento actual a analizar.
  - Observa ($\mathbf{S_m}$) /* *S_m contiene el estado actual del DFA* */
  - Acción := TAcción [S_m , Nexttoken].
  - SI (Acción = **ACC**) ENTONCES:
    - Aceptar el string de entrada.
    - Terminar el análisis.
  - SI (Acción = **ERR**) ENTONCES:
    - Existe un error en la entrada.
    - Llamar a la rutina de manejo de errores.
  - SI (Acción = **sN**) ENTONCES:
    - Insertar Nexttoken en la Pila.
    - Insertar N en la Pila. /*N es el nuevo estado del DFA */
    - Actualizar el valor de Nexttoken (obtener el siguiente).
  - SI (Acción = **rM**) ENTONCES:
    - Sea (**A → ß**) la producción número M de la gramática.
    - Sacar (2 * **l ß l**) símbolos de la Pila. (|ß| = longitud de ß)
    - Observa($\mathbf{S_m}$) /* el elemento que ahora está en el tope de la pila*/
    - Insertar a A en la pila.
    - Insertar en la pila K /* K es el # de estado almacenado en TGoto[S_m, A] */

CONSTRUCCION DE LA MATRIZ SLR

Definiciones necesarias para la construcción de la matriz SLR:

La idea central del método SLR es construir un DFA a partir de la gramática y llenar una Matriz.

1. Elementos SLR (ITEM)
Un ITEM SLR es una producción de la gramática con un apuntador en alguna parte de su lado derecho. Esto es un ITEM SLR es de la forma $A \rightarrow \beta \bullet \alpha$.
Por ejemplo, de la producción $A \rightarrow XY$ se pueden generar los siguientes ITEMS :

$A \rightarrow \bullet XY$
$A \rightarrow X \bullet Y$
$A \rightarrow XY \bullet$

De la producción $A \rightarrow \varepsilon$ sólo puede generarse un item $A \rightarrow \varepsilon\bullet$.
Un ITEM SLR indica hasta qué punto de la producción se ha "aceptado" en cierto momento del proceso de análisis.

2. Estados SLR.
Un estado SLR es un conjunto de ITEMS SLR. Por ejemplo un posible estado SLR podría ser :
$\{A \rightarrow a\bullet Bd \ , \ B \rightarrow \bullet d \ , \ B \rightarrow \varepsilon\bullet\}$. Un estado SLR se representa como : I_i.

- Para construir la matriz SLR se requiere utilizar una GRAMATICA AUMENTADA (**G'**) y 2 funciones CERRADURA (closure) y GOTO.

3. Gramática Aumentada (G').
Se le da el nombre de gramática aumentada a la gramática G que tiene un nuevo símbolo inicial (**G'**) y una producción adicional $G' \rightarrow G$. Este nuevo símbolo inicial sirve para indicarle al analizador cuando debe detenerse y aceptar la entrada. Este significa que un string de entrada es aceptado únicamente cuando el analizador reduce la producción $G' \rightarrow G$.

Definición formal:
Sea G = { N, T, P, S } la gramática a la cual se le desea construir un analizador SLR, entonces se debe diseñar la gramática $G' = \{ N', T, P', S' \}$ donde $N' = N \cup \{S'\}$ y $P' = P \cup \{ S' \rightarrow S \}$.

4. Función Cerradura.
Si I es un conjunto de ITEMS de la gramática G, entonces la cerradura (I), es el conjunto de items construído a partir de I utilizando las siguientes reglas:

1). Todos los ITEMS en I se añaden a la cerradura (I).
2). Si $A \rightarrow \alpha \bullet B\beta$ está en la cerradura (I) y $B \rightarrow \gamma$ es una producción , entonces añadir el ITEM $B \rightarrow \bullet\gamma$ a I, si todavía no está ahí. Esta regla se aplica hasta que no puedan añadirse más ITEMS a la cerradura(I).

5. Operación GOTO(I , X).
La cerradura del conjunto de todos los ITEMS $A \rightarrow \alpha X \bullet \beta$ tal que $A \rightarrow \alpha \bullet X\beta$ está en I. Donde I es un conjunto de ITEMS y X es un símbolo gramatical(Terminal ó No-Terminal).

CONSTRUCCION DEL CONJUNTO DE ESTADOS SLR.

El algoritmo que se requiere para la construcción del conjunto de estados para el analizador SLR utilizando una gramática aumentada (G') quedaría como:

PROCEDURE ITEMS(**G'**);
BEGIN
C := { CERRADURA({**S' → S**}) };
REPETIR
Para cada conjunto de ITEMS I en C y para cada símbolo gramatical X
tal que GOTO (I,X) exista (no esté vacío) y no esté en C hacer
Añadir GOTO (I,X) a C.
HASTA que no se puedan añadir más conjuntos de ITEMS a C.
END.

Una vez que se tienen todos los estados (del DFA), se puede definir el algoritmo que sirve para la construcción de las tablas de ACCION y BRINCO de la matriz SLR. Para esto se requiere conocer los FOLLOW(A) para cada símbolo No-Terminal A de la gramática.

ALGORITMO PARA LA CONSTRUCCION DE LA MATRIZ S.L.R.

Entrada: Una gramática aumentada G'
Salida: Las tablas de ACCION y BRINCO de la matriz SLR.
Proceso:

1. Construir el conjunto de estados SLR para G' C = { I_0, I_1, I_2,... I_n }
2. El estado i de la tabla se construye a partir de I_i . Las acciones correspondientes al estado i se determinan como sigue:
   a). Si $A \rightarrow \alpha \cdot a\beta$ está en I_i y GOTO(I_i, a) = I_j , entonces añadir **shift j** a la tabla de acción en ACCION [i, a]. Esto ocurre sí y sólo si, a es un símbolo terminal.
   b). Si $A \rightarrow \alpha \cdot$ está en I_i, entonces añadir **reduce** $A \rightarrow \alpha \cdot$ en la tabla de acción en ACCION [i,a] para todos los símbolos terminales del FOLLOW(A), excepto para cuando A = S'.
   c). Si $S' \rightarrow S \cdot$ está en I_i , entonces añadir **ACC** en la tabla de acción en ACCION[i,$]
3. Las transiciones GOTO para el estado i se construyen para todos los símbolos No-Terminales A utilizando la siguiente regla:
   Si GOTO(I_i , A) = I_j , entonces añadir j en la tabla de goto en GOTO[i, A].
4. Todas las casillas que hayan quedado sin definir representan un estado de error.
5. El estado inicial del analizador es áquel construido a partir de $S' \rightarrow \cdot S$.

NOTA:

- Puede darse el caso de una casilla que contenga simultaneamente un Shift y un Reduce (Porque la gramática no sea estrictamente SLR). Cuando esto suceda se deberá elegir la acción de Shift y desechar la de Reduce.

L.R. CANÓNICO

1. Elementos LR(1) (ITEM)

Un ITEM LR(1) es una producción de la gramática con un apuntador en alguna parte de su lado derecho y con un terminal(es) (lookahead) asociado a él. Esto es un ITEM LR(1) es de la forma $A \Rightarrow \beta \bullet \alpha, a$.

Donde $A \Rightarrow \beta \bullet \alpha$ es una producción de la gramática y a es un símbolo terminal ó el símbolo $.

En los ITEM´s LR(1), el 1 se refiere a la longitud del segundo componente (a) que es el lookahead.

Este lookahead no tiene ningún efecto en ITEMS de la forma $A \Rightarrow \beta \bullet \alpha, a$; donde $\alpha <> \varepsilon$, pero en ITEMS de la forma $A \Rightarrow \alpha \bullet, a$; significa reducir la producción $A \Rightarrow \alpha \bullet$ si el siguiente símbolo de entrada es a.

2. Estado LR(1) es un conjunto de ITEMS LR(1). Un estado LR(1) se representa como : I_i.

3. Gramática Aumentada (G') se maneja el mismo concepto que en el SLR.

4. Función Cerradura. La función de cerradura para el método LR Canónico, se define como:

```
FUNCTION CERRADURA (I);
  REPETIR
      Para cada ITEM [A ⇒ α •Bβ , a] en I ,
          Para cada producción B ⇒ γ
              Para cada terminal b en FIRST(βa) tal que [ B ⇒ • γ , b ] no está en I hacer
                      Añadir [ B ⇒ • γ , b ] a I
  HASTA que no se puedan añadir más items a I .
```

5. Operación GOTO(I , X). La operación se define como:

```
FUNCTION GOTO (I , X);
  Sea J el conjunto de ITEMS [A ⇒ αX •β , a] tal que [A ⇒ α •Xβ , a] está en I hacer
      GOTO := CERRADURA ( J );
```

6. CONSTRUCCION DEL CONJUNTO DE ESTADOS LR(1).

```
PROCEDURE ITEMS(G');
  C := { CERRADURA( {S' ⇒ S , $} ) };
  REPETIR
    Para cada conjunto de ITEMS I en C y para c/ símbolo gramatical X tal que GOTO (I ,X)
    exista (no esté vacío) y no esté en C hacer
        Añadir GOTO (I ,X) a C.
  HASTA que no se puedan añadir más conjuntos de ITEMS a C.
```

7. CONSTRUCCION DE LA MATRIZ L.R. CANONICA

Se utiliza el mismo algoritmo que en el método SLR, la única diferencia es al momento de colocar las REDUCCIONES, ya que sólo se colocarán en el (los) look-aheads que tenga asociados la producción.

L.A.L.R.

El método LALR (lookahead-LR) es la técnica bottom-up más comúnmente utilizada, debido a que el tamaño de la matriz que se genera es, generalmente, bastante más pequña que la que se obtiene del método LR Canónico. Adicionalmente se sabe que este método funciona para la mayoría de las construcciones sintácticas.

Para construir la matriz LALR:

1° Se obtienen todos los estados por el método LR Canónico.

2° Se mezclan aquellos estados que tengan la misma "cerradura" pero diferentes lookaheads. De esto se obtiene un estado con esa cerradura y con la unión de los lookaheads involucrados.

3° Al final, se construye la matriz LALR usando el mismo algoritmo que se empleó para el LR Canónico.

ANÁLISIS SEMÁNTICO Y GENERACIÓN DE CÓDIGO INTERMEDIO

ANÁLISIS SEMÁNTICO.

El análisis de semántica en el área de lenguajes de programación incluye algunas verificaciones tales como: Existencia y Unicidad de Variables, Compatibilidad de Tipos de Datos, Congruencia en cantidad de parámetros, etc. Este parte del análisis se lleva a cabo, no como una etapa independiente del proceso de traducción, sino como reglas que se distribuyen a lo largo del proceso de traducción. Algunas técnicas existentes para desarrollar el análisis semántico se discutirán más adelante en esta guía.

GENERACIÓN DE CÓDIGO INTERMEDIO

El tema principal de esta guía es la fase de ***generación de código intermedio*.**
Como ya se sabe, el código intermedio forma un lenguaje de bajo nivel, sin llegar al nivel más primitivo.

Tipos de Código Intermedio

Existen diversos formatos para representar las instrucciones en código intermedio. Se presentarán para mostrar las alternativas que existen, pero el ***diseñar*** o ***elegir*** un formato de código intermedio será trabajo del programador del traductor.
En los ejemplos que aparecerán, se ha dejado el identificador (nombre) de la variable, pero lo que realmente debe aparecer es la dirección generada por el análisis de léxico, ya que uno de los objetivos de esa fase es precisamente asociar una direccion única a cada una de las variables.

Notación Polaca

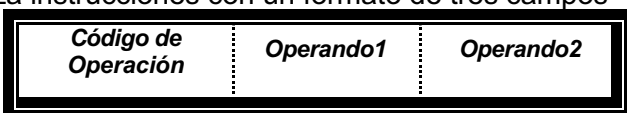
Este tipo de código intermedio sirve para pasar de una notación de infijo a una notación de postfijo. A continuación se mostrará un ejemplo de este tipo de traducción : **A := B + C * D**
Esta instrucción convertida a notación polaca queda : **A B C D * + :=**

Esta notación resuelve el orden en que se deben de ejecutar las operaciones de acuerdo a la prioridad de los operadores.
Para ejecutar este tipo de código se requiere del uso de una pila de ejecución, cuyo algoritmo es el siguiente :

```
i = 1
REPETIR
 SI vector_polaco [ i ] = variable ENTONCES
       PUSH pila_de_ejecución(variable)
 SI NO
   SI vector_polaco [ i ] = operador ENTONCES
        POP pila_de_ejecución ¡ elemento_1
        POP pila_de_ejecución ¡ elemento_2
        PUSH pila_de_ejecución(elemento_1 operador elemento_2)
   SI NO
      .....(*diversas operaciones, ajenas a expresiones*)
 i = i + 1
 HASTA fin de vector_polaco
```

Triplos

Este tipo de código utiliza instrucciones con un formato de tres campos



A continuación se mostrará un ejemplo de este tipo de traducción :

A := B + C * D

Esta instrucción convertida a triplos queda :

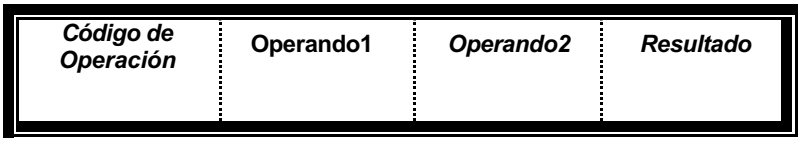
```
*    C   D
+    B
:=   A
```

La ejecución de los triplos también requiere de una pila. El algoritmo para ejecutar triplos es :

```
PARA cada triplo HACER
   SI está explícito el operando_1 y el operando_2 ENTONCES
         PUSH pila_de_ejecución (operando_1 operador operando_2)
   SI sólo está explícito el operando_1 ENTONCES
      PUSH pila_de_ejecución (POP pila_de_ejecución operador operando_1 )
      ..... (*diversas operaciones*)
   SI no está explícito ningún operando ENTONCES
     PUSH pila_de_ejecución (POP pila_de_ejecución operador POP pila_de_ejecución )
```

Cuádruplos

Este tipo de código utiliza instrucciones con un formato de cuatro campos



A continuación se mostrará un ejemplo de este tipo de traducción :

A := B + C * D

Esta instrucción convertida a cuádruplos queda :

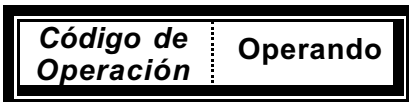
```
*    C     D     T1
+    B     T1    T2
:=  T2           A
```

Donde T1 y T2 son direcciones temporales, seleccionadas por el traductor; aunque para generar este tipo de código el traductor debe efectuar más procesamiento, el ejecutador se ve beneficiado porque el algoritmo de ejecución queda muy simple, y se muestra a continuación:

```
PARA cada cuádruplo HACER
   SI el operador es binario ENTONCES
         Resultado := operando1 operador operando2
    ..... (*diversas operaciones*)
```

Código P

La máquina P es una máquina emulada por Software ideada por Niklaus Wirth (creador del Pascal) y preparada para ejecutar código P. El código P requiere para su ejecución de una pila, las instrucciones en P hacen referencia a esta pila.
El haber diseñado esta máquina y el haber pensado en que el Pascal se tradujera a este tipo de código permitió acortar considerablemente el tiempo de desarrollo de este traductor, y lo convirtió en un traductor transportable pero con la desventaja que queda lenta la ejecución de un programa traducido de esta manera. El formato de las instrucciones en P es el siguiente:



Si se omite el operando, el código de operación hace referencia al top y al top-1 de la pila de ejecución.
A continuación se mostrará un ejemplo de este tipo de traducción :

A := B + C * D

Esta instrucción convertida a código P queda :

Carga B
Carga C
Carga D
Multiplica
Suma
Almacena A

En este ejemplo se utilizaron códigos de operación generales, pero realmente en código P existen códigos especiales para hacer referencias a variables locales o globales.

Comparación de los diversos métodos expuestos

En cuanto a la cantidad de memoria que requieren para su almacenamiento, podríamos ordenarlos de menor a mayor de la siguiente manera:

- Notación polaca
- Código P
- Triplos
- Cuádruplos

En cuanto a velocidad de su ejecución, podríamos ordenarlos de menor a mayor como sigue:

- Cuádruplos
- Triplos, Código P
- Notación polaca

Si lo que se desea es convertir el código intermedio a código objeto, ordenando de menor a mayor grado de complejidad quedaría:

- Cuádruplos
- Triplos, Código P
- Notación polaca

Se debe efectuar la decisión sobre el tipo de código que conviene generar, o si requiere diseñar un nuevo tipo de formato que cubra sus necesidades de traducción.

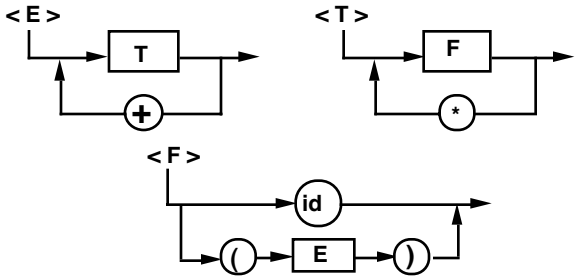
Una gran ventaja de los métodos que vamos a estudiar para generar código intermedio, es que su implementación podrá ser modular, entonces, aunque todavía no se entienda el método completo, podrá comenzarse diseñando un primer módulo que podría ser el de expresiones aritméticas.

Generación de Código Intermedio para expresiones aritméticas

Diagrama de Sintaxis para las Expresiones

Aunque sintácticamente, es totalmente equivalente colocar todos los operadores en un mismo nivel o jerarquía, dado que la generación de código intermedio será controlada por el análisis sintáctico, entonces nos convendrá separar los diagramas, y asimismo la gramática, por prioridad de los operadores.
En Pascal no se hace distinción entre las expresiones aritméticas o booleanas, todas pertenecen a la clase EXPRESION, pero nosotros tomaremos primeramente un subconjunto de las expresiones que sólo contenga a algunos de los operadores aritméticos.

A continuación se muestra el subconjunto de las expresiones que utilizaremos para diseñar sus acciones de generación de código.



Evaluación de una expresión

Una expresión puede ser evaluada de derecha a izquierda, o de izquierda a derecha, a esto se le llama asociatividad derecha e izquierda, respectivamente.

Asociatividad derecha

Usando asociatividad derecha la evaluación de la siguiente expresión mostrada a continuación se realizaría de la siguiente manera:

3 + 2 + 5
3 + 7
10

Este tipo de asociatividad no es válida ni para resta ni para la división.

Asociatividad izquierda

Usando asociatividad izquierda la evaluación de la siguiente expresión mostrada a continuación se realizaría de la siguiente manera:

3 + 2 + 5
5 + 5
10

Este tipo de asociatividad es válida para todas las operaciones.

Generación de Notación Polaca para asociatividad derecha

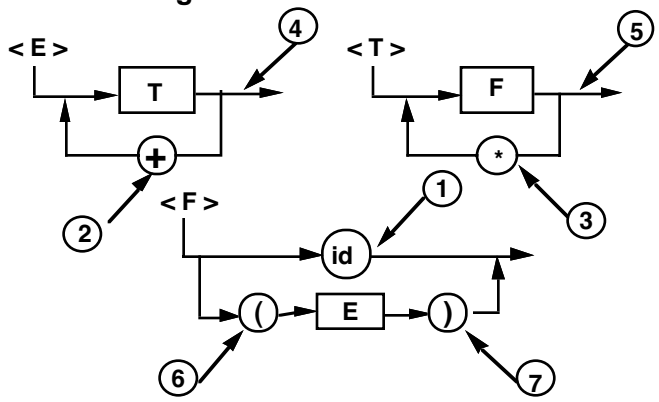
Mostraremos las acciones de generación de código directamente sobre los diagramas de sintaxis, pensando en que será muy sencillo reconocer su localización sobre el programa de análisis sintáctico.

Para generar código en notación polaca, al que llamaremos ***vector polaco*** requeriremos:

- ***Pila de Operadores***
- ***Vector para almacenar el código***

Se podrá observar lo sencillo que es introducir las acciones al diagrama de sintaxis, y se podrá notar que de esta manera se automatizará el proceso de traducción de las expresiones.

Acciones de generación de código



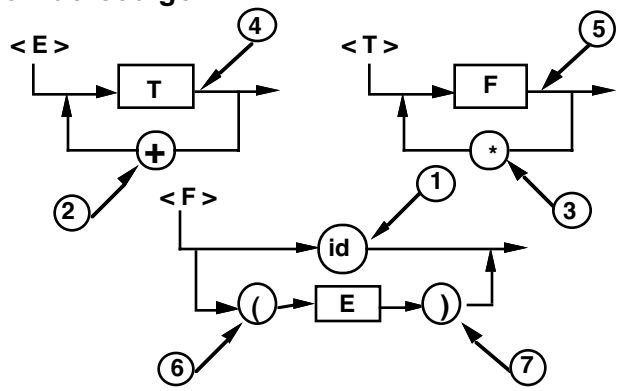
1.- Escribir en el vector polaco, la dirección de la variable generada por el análisis de léxico.
2.- Push pila-de-operadores(+)
3.- Push pila-de-operadores(*)
4.- MIENTRAS el top de la pila de operadores contenga un **+** HACER LO SIGUIENTE:
vector polaco (dirección actual) = pop pila-de-operadores
5.- MIENTRAS el top de la pila de operadores contenga un ***** HACER LO SIGUIENTE:
vector polaco (dirección actual) = pop pila-de-operadores
6.- Push pila-de-operadores(marca de fondo falso)
7.- Pop pila-de-operadores.... se quita la marca de fondo falso

Para probar este método se recomienda marcar los diagramas y utilizar una pila para seguirlos.

Generación de Notación Polaca para asociatividad izquierda

Como se habrá notado en los ejercicios anteriores, la pila de operadores acumulará todos los operadores de igual prioridad que aparezcan en la expresión a traducir; para asociatividad izquierda a lo más se requerirá acumular un operador.
Las acciones de generación de código quedan muy similares, sólamente se debe mover de posición a la acción **4** y a la acción **5**.

Acciones de generación de código



1.- Escribir en el vector polaco, la dirección de la variable generada por el léxico.
2.- Push pila-de-operadores(+)
3.- Push pila-de-operadores(*)
4.- SI el top de la pila de operadores = + ENTONCES
vector polaco (dirección actual) = pop pila-de-operadores
5.- SI el top de la pila de operadores = * ENTONCES
vector polaco (dirección actual) = pop pila-de-operadores
6.- Push pila-de-operadores (marca de fondo falso)
7.- Pop pila-de-operadores.... se quita la marca de fondo falso

Cabe hacer notar que por el momento se está asumiendo que todas las variables son **globales**,

Acciones de generación de código en la gramática

Si convertimos directamente los diagramas de sintaxis del subconjunto de expresiones aritméticas a una gramática, podríamos obtener:

E → T + E
E → T
T → F * T
T → F
F → id
F → (E)

Si el método de reconocimiento sintáctico que vamos a seguir es un método top-down, entonces la gramática se deberá transformar, y quedaría de la siguiente manera:

E → TE'
E' → + T E'
E' → ε
T → FT'
T' → *F T'
T' → ε
F → id
F → (E)

Colocando las acciones de generacción de código sobre esta gramática obtenemos para:

Asociatividad izquierda

E → **TE'**
E' → + {acción2} **T** {acción4} **E'**
E' → ε
T → **FT'**
T'→ * {acción3} **F** {acción5} **T'**
T'→ ε
F → **id** {acción1}
F → **(**{acción6} **E)** {acción7}

1.- Escribir en el vector polaco, la dirección de la variable generada por el análisis de léxico.
2.- Push pila-de-operadores(+)
3.- Push pila-de-operadores(*)
4.- SI el top de la pila de operadores = + ENTONCES
vector polaco (dirección actual) = pop pila-de-operadores
5.- SI el top de la pila de operadores = * ENTONCES
vector polaco (dirección actual) = pop pila-de-operadores
6.- Push pila-de-operadores (marca de fondo falso)
7.- Pop pila-de-operadores.... se quita la marca de fondo falso

Generación de cuádruplos.

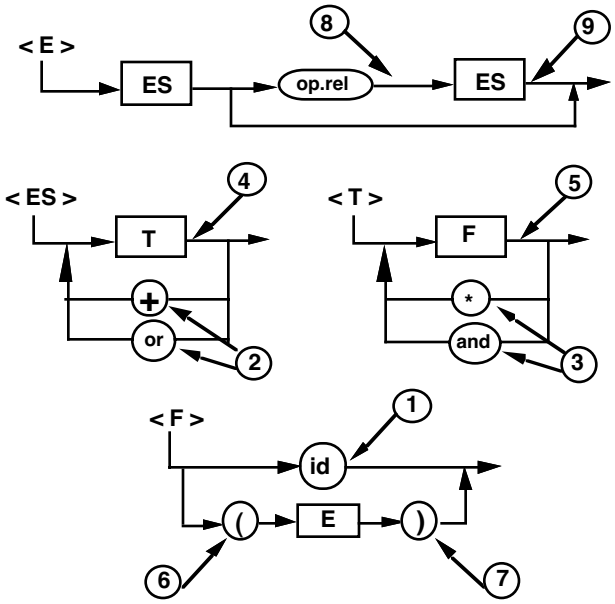
A partir de este momento, sólo consideraremos a la asociatividad izquierda, ya que es la única válida para todas las operaciones. Para generar cuádruplos requeriremos:

- ***Pila de operadores***
- ***Pila de operandos***
- ***Avail de direcciones temporales***
- ***Espacio para almacenar los cuádruplos generados***

Como las acciones no cambian colocándolas en los diagramas de sintaxis o en la gramática, de aquí en adelante sólo las colocaremos en los diagramas de sintaxis.

ACCIONES DE GENERACIÓN DE CÓDIGO PARA EXPRESIONES COMPLETAS, INCLUYENDO OPERADORES BOOLEANOS

Para que los diagramas de expresiones queden completos, sólo nos falta añadir a los operadores booleanos, las acciones no sufren cambios, sólo faltan las de los operadores relacionales. Mostraremos de nuevo un subconjunto de las expresiones, tomando algún operador ejemplo para cada nivel de prioridad.

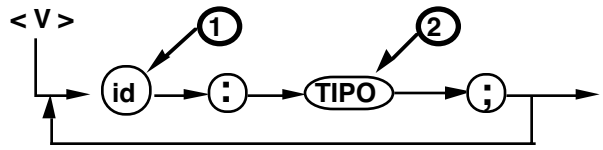


1.- Push pila-de-operandos(dirección de la variable)
2.- Push pila-de-operadores(operador)
3.- Push pila-de-operadores(operador)
4.- SI el top de la pila de operadores **= +, or** ENTONCES
Generar cuádruplo: **operador operando1 operando2 Resultado** donde,
operando2 = Pop pila-de-operandos
operando1 = Pop pila-de-operandos
Resultado = Temporal obtenido del avail
SI alguno de los operandos correspondía a un temporal ENTONCES Regresarlo al avail
Push pila-de-operandos(Resultado)
Pop pila-de-operadores.
5.- SI el top de la pila de operadores **= *,and** ENTONCES
**** igual a acción 4**
6.- Push pila-de-operadores (marca de fondo falso)
7.- Pop pila-de-operadores.... se quita la marca de fondo falso
8.- Push pila-de-operadores (operador)
9.- Generar cuádruplo: **operador operando1 operando2 Resultado** donde,
operando2 = Pop pila-de-operandos
operando1 = Pop pila-de-operandos
Resultado = Temporal obtenido del avail
SI alguno de los operandos correspondía a un temporal ENTONCES Regresarlo al avail
Push pila-de-operandos(Resultado)
Pop pila-de-operadores.

Como se habrá notado, los diagramas mostrados permiten algunas secuencias que serán imposibles de evaluar, y además, en aquéllas que tienen posibilidad de evaluarse, se requiere de una combinación específica de tipos para las variables. Por lo tanto las expresiones requieren de un ***análisis semántico*** tema que será tratado en la siguiente sección.

ANÁLISIS SEMÁNTICO

Para efectuar el análisis semántico de una expresión se revisará la combinación de tipos a los que pertenezcan las variables y constantes que actúen como operandos.
Para incluir los tipos (simples, más tarde revisaremos los dimensionados) de las variables en la tabla de símbolos podemos efectuar las siguientes acciones:



1.-Push pila-de-operandos (dirección de la variable)
2.-Poner el **tipo** a todas las variables que se metieron a la pila de operandos, y sacarlas de la pila.

Reglas semánticas

Utilizaremos como referencia las reglas semánticas de las expresiones en Pascal, para esto construiremos una tabla donde

E = entero **R = real** **C = caracter**
S = string **B = booleano** **X = ERROR SEMANTICO**

op1	op2	*,+,-	/	div mod	relac	and or
E	E	E	R	E	B	X
E	R	R	R	X	B	X
R	E	R	R	X	B	X
R	R	R	R	X	B	X
C	C	X	X	X	B	X
C	S	X	X	X	B	X
S	C	X	X	X	B	X
S	S	X	X	X	B	X
B	B	X	X	X	B	B

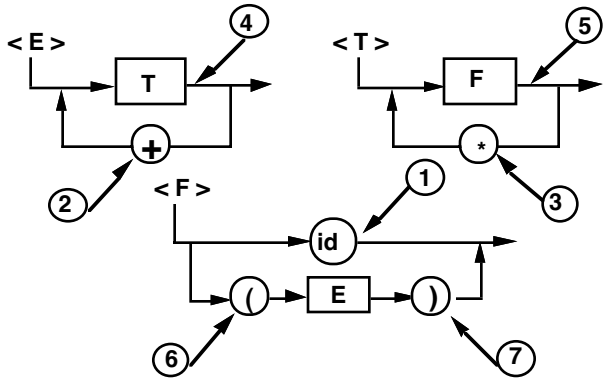
En esta tabla se omitieron un conjunto de combinaciones que con cualquier operación producer error, como por ejemplo ENTERO con STRING.
Ya que un traductor es una autómata (con funcionamiento automático), es conveniente que el análisis semántico también se automatice. Las claves para automatizar el análisis semántico son:

- Utilizar acciones para la verificación semántica
- Escoger una estructura de datos que permita accesarla directamente, encontrar el resultado de una operación y descubrir si ésta es o no válida.

De hecho estas claves están vigentes para todo el proceso de traducción, en cuanto a utilizar acciones y estructuras de datos automáticas.
Para el análisis semántico no se recomienda usar la tabla mostrada anteriormente, debido a que su acceso no es automático y le faltan muchas combinaciones.

ACCIONES PARA VERIFICACIÓN SEMÁNTICA

Para mostrar un ejemplo de como diseñar las acciones de verificación semántica, utilizaremos un subconjunto de expresiones aritméticas. Estas acciones se deberán de **añadir** a las acciones de generación de código, y no modifican nada de lo ya visto en generación de código.
Para realizar la verificación semántica se requerirá de una pila de tipos.



1.- Push pila-de-tipos (tipo de la variable)
2, 3 No llevan acción semántica
4.- SI tipos del top y top-1 de la pila de tipos son permitidos en la operación a generar ENTONCES
Pop pila-de-tipos; Pop pila-de-tipos
Push pila-de-tipos (resultado de la operación)
SI NO
Marcar error semántico, y aplicar acción correctiva que podría ser:
Pop pila-de-tipos; Pop pila-de-tipos
Push pila-de-tipos (posible resultado de la operación)
5.- Igual a 4.
6, 7 No llevan acción semántica

GENERACIÓN DE CÓDIGO PARA ESTATUTOS

ESTATUTOS CONDICIONALES

Para los estatutos que traduciremos a partir de esta sección, se requerirá de una nueva pila, la **pila de saltos**, además introduciremos una nueva instrucción en código intermedio, la instrucción **goto** que aparecerá en varias modalidades:

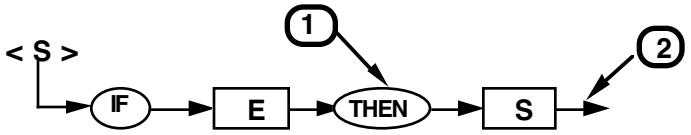
goto	*(* incondicional *)*
gotofalso operando	*(* que efectúa un salto si el operando tiene un valor de falso*)*
gotoverdadero operando	*(* que efectúa un salto si el operando tiene un valor de verdadero*)*

Generalmente, al generar un goto, aún no sabemos a qué dirección saltará, entonces quedará pendiente por rellenar; para efectuar esta función utilizaremos al procedimiento:

rellenar(dirección a rellenar, valor con que se rellena)

Además utilizaremos un contador que llamaremos **cont** que contendrá la dirección del siguiente cuádruplo a generar, al inicio de la generación de código, cont tendrá un valor de 1.

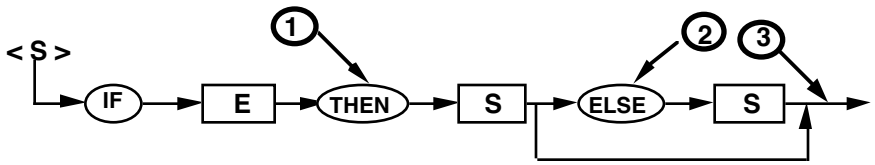
Estatuto IF-THEN



1.- aux = POP PTipos
SI **aux** diferente de booleano ENTONCES **error semántico.**
sino
Sacar **resultado** de PilaO
Generar **gotofalso resultado** _____
PUSH PSaltos (cont-1)

2.- Sacar **fin** de PSaltos
rellenar (**fin**, cont)

Estatuto IF-THEN-ELSE



1.- aux = POP pila-de-tipos
SI aux diferente de booleano ENTONCES **error semántico.**
Sino
Sacar **resultado** de pila-de-operandos
Generar **gotofalso resultado** _____
PUSH pila-de-saltos (cont)

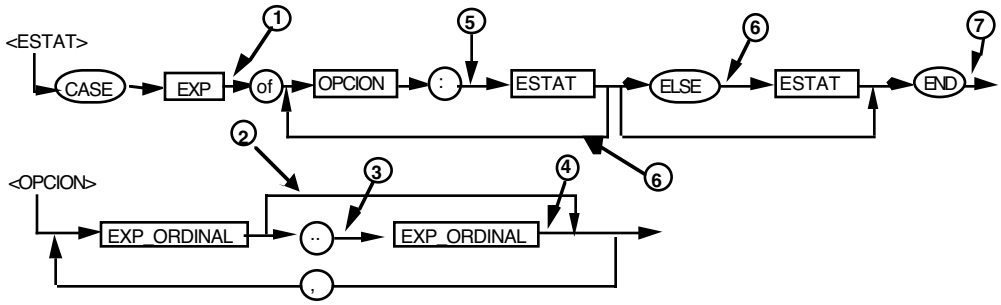
2.- Generar **goto** _____
sacar **falso** de pila-de-saltos
rellenar (rellenar **falso**, cont)
PUSH pila-de-saltos (cont - 1)

3.- Sacar **fin** de pila-de-saltos
rellenar (**fin**, cont)

NOTA: Para efectos prácticos:

- ***pila-de-operandos = PilaO***
- ***pila-de-operadores = Poper***
- ***pila-de-saltos = Psaltos***
- ***pila-de-tipos = PTipos***

Estatuto CASE



1.- Verificar que el tipo de la expresión sea Ordinal (entero, char, bool, ...)
Meter una marca de fondo falso en la PilaSaltos.

2.- Verificar que la expresión ORDINAL (que debe ser un dato simple) tenga el mismo tipo que EXP (el tope actual de la PilaO.
Cte = Pop de la PilaO, Exp= Pop de la PilaO
Generar el cuádruplo : = , Exp, Cte, Tk
Generar el cuádruplo : GotoV, Tk, ____
Sacar el temporal Tk de la PilaO.
Meter nuevamente Exp a la PilaO
Meter CONT-1 en la PilaSaltos.

3.- Verificar que la expresión ORDINAL (que debe ser un dato simple) tenga el mismo tipo que EXP (el tope actual de la PilaO.
Cte = Pop de la PilaO, Exp= Pop de la PilaO

Meter nuevamente Exp a la PilaO
Generar el cuádruplo : >= , Exp, Cte, Tk

4.- Verificar que la expresión ORDINAL (que debe ser un dato simple) tenga el mismo tipo que EXP (el tope actual de la PilaO.
Cte = Pop de la PilaO, Exp= Pop de la PilaO
Generar el cuádruplo : <= , Exp, Cte, Tj
Sacar el temporal Tj de la PilaO.
Sacar el temporal Tk de la PilaO
Generar el cuádruplo : AND, Tj, Tk, Tn
Generar el cuádruplo : GotoV, Tn, ____
Meter CONT-1 en la PilaSaltos.

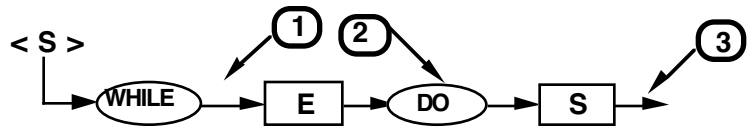
5.- Mientras no se encuentre la marca de fondo falso de PilaSaltos, sacar y:
Rellenar todos los GotoV con CONT+1
Generar un Goto, ____
Guardar CONT-1 en la PilaSaltos.

6.- Rellenar el Goto que está en el tope de la PilaSaltos con CONT+1
Generar un Goto, ____
Guardar CONT-1 en la PilaSaltos.

7.- Sacar el tope de la PilaO (es Exp).
Mientras existan Goto pendientes en PilaSaltos,
Rellenar cada uno con CONT

ESTATUTOS DE REPETICIÓN

Estatuto WHILE

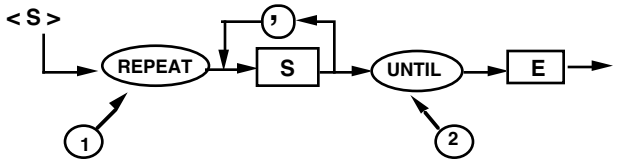


1.- Meter **cont** en PSaltos

2.- Sacar **aux** de PTipos
SI **aux** diferente de booleano ENTONCES **error semántico.**
sino
Sacar **resultado** de PilaO
Generar **gotofalso resultado** _____
PUSH Psaltos (cont-1)

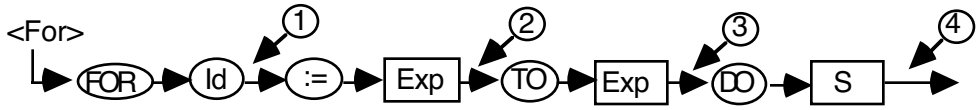
3.- Sacar **falso** de PSaltos. Sacar **retorno** de PSaltos
Generar **goto retorno**
rellenar (**falso**, cont)

Estatuto REPEAT



1.- PUSH pila-de-saltos (cont)
2.- Generar **gotofalso POP pila-de-operandos POP pila-de-saltos**

Estatuto FOR



1.- Guardar el identificador (dirección) en pila de Operandos (PilaO), verificar semántica.
2.- Exp1 = Pop de PilaO (es la variable que contiene el "resultado" de la expresión).
Id= Tope de la Pila de operandos (PilaO) (sin sacarlo).
Generar el cuádruplo : (:= , Exp1, , Id)
3.- Obtener una variable Temporal (Tf) /* Ver nota */
Exp2= Pop de la PilaO (es la variable que contiene el "resultado" de la expresión2).
Obtener otra variable temporal (Tx)
Generar los cuádruplos:
(:= , Exp2, , Tf)
(<=, Id , Tf, Tx) /* Sin liberar a Tf */
(Gotof, Tx, ___)
Liberar la variable temporal Tx.
Meter en la PilaSaltos dirección cont-2 (#de cuádruplo del <=)
4.- Id = Pop de PilaO
Genera cuádruplo: (+ , Id , 1 , Id)
retorno = Pop de PilaSaltos.
Genera cuáduplo: (Goto , retorno)
Rellena (retorno +1 , Cont) /* Corresponde al GotoF*/
Libera la variable temporal Tf.

NOTA: Se utiliza una variable temporal Tf para dejar el resultado de Exp2, porque si Exp2 fuera una variable simple (ej: N), se podría alterar su valor dentro de algún estatuto del For y pudiera ocasionar problemas. (ej: provocar un ciclo infinito).

GENERACIÓN DE CÓDIGO PARA VARIABLES DIMENSIONADAS

El lugar de trabajo para las variables simples podrá coincidir en la dirección que se les generó en el análisis de léxico, pero en las variables dimensionadas, se requerirá de un mayor espacio de trabajo.
Si el lenguaje de programación lo permite, el usuario podrá declarar variables de cualquier dimensión, y es prácticamente imposible que el traductor haya contemplado dentro de su definición estructuras de datos para aceptar cualquier dimensión de cualquier orden. Así que lo que puede hacer el traductor es convertir los arreglos (a traducir) de cualquier dimensión en arreglos de una dimensión.
Para esto hay varios procedimientos, el que utilizaremos es el siguiente:

Dada una declaración de variable dimensionada

id-dim : **array [límite inferior1..límite superior1, límite inferior2..límite superior2, ... , límite inferiorn .. límite superiorn] of tipo**

El cálculo para una encontrar la dirección en una dimensión se puede realizar como
id-dim[s_1,s_2,...s_n] =

(s_1-límite inferior1)*d2*d3*....*dn + (s_2 - límite inferior2) *d3*d4*.... * dn + ...
+ (s_{n-1}- límite inferiorn-1) *dn + (s_n- límite inferiorn) + DirecciónBASE

donde
di = límite superior i - límite inferior i + 1

Si se generaran todas las operaciones involucradas en la fórmula anterior, se requeriría de mucho tiempo de ejecución para encontrar la dirección en una dimensión, así que rearreglaremos esta fórmula agrupando todas las constantes que puedan calcularse desde la declaración de la variable.

De esta manera la fórmula queda:

id-dim[s_1,s_2,...s_n] = s_1*m_1 + s_2*m_2 + ... + s_{n-1}*m_{n-1} + s_n + K + BASE

donde
m_1= d2*d3*....*dn
m_2= d3*d4*....*dn
.....
m_{n-1}= dn

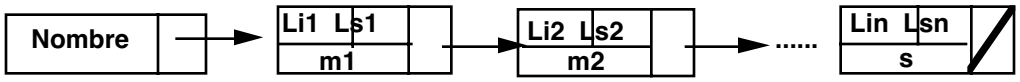
K = - (límite inferior1 * m_1 + límite inferior2 * m_2 + ...+ límite inferior$_n$)

Dejar así la fórmula reduce notablemente las operaciones a generar para traducir la referencia de la variable dimensionada, ya que el cálculo de las **m_i** y **S** se efectúa sólamente en el momento de la declaración de la variable y no cada vez que se hace referencia a ella.

Aunque existen otras alternativas para traducir la referencia de una variable dimensionada, nos concentraremos en ésta porque es la que produce menor número de operaciones, y por tanto, mayor velocidad de ejecución.

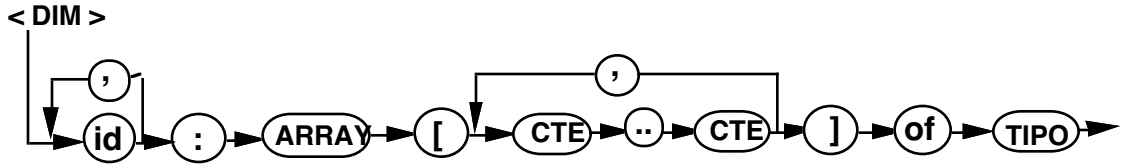
Para efectuar la declaración de una variable dimensionada, si es que se va a utilizar la fórmula reducida, es conveniente almacenar en la misma estructura en la que se va guardar la descripción de las dimensiones el valor de las constantes **m** y **S.**

Una posible estructura de datos para efectuar la declaración y almacenarla en la tabla de símbolos sería:
Tabla de símbolos



Además se deberá almacenar la **BASE** relacionada con el nombre del id.

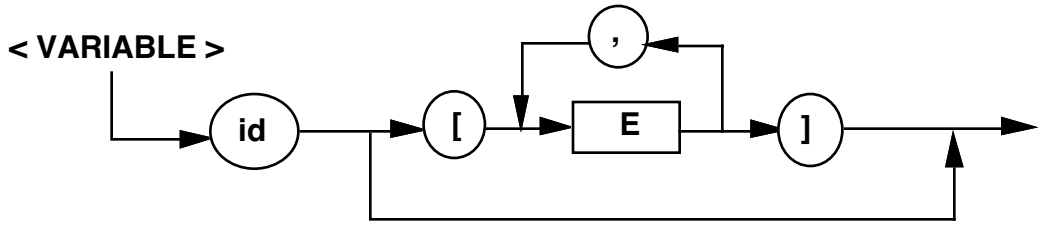
Declaración de VARIABLES DIMENSIONADAS



1.- Guardar dir de id en PilaO
2.- Indicar que las variables son dimensionadas.
3.- Obtener un campo para la descripción de la dimensión.
Ligar todos los identificadores de la pila con este campo.
DIM = 1, R = 1
4.- Almacenar, en campo de descripción, la constante en Li_{DIM}.
5.- Almacenar, en campo de descripción, la constante en Ls_{DIM}.
$R = (Ls_{DIM} - Li_{DIM} + 1) * R$
6.- DIM = DIM + 1
Obtener un nuevo campo para la siguiente descripción.
Ligar el campo anterior con el nuevo campo.
7.- Ligar el último campo con nulo.
Regresar al primer campo de descripción
DIM = 1, SUMA = 0 , AUX = R
REPETIR
$m_{DIM} = R / (Ls_{DIM} - Li_{DIM} + 1)$
$R = m_{DIM}$
$SUMA = SUMA + Li_{DIM} * m_{DIM}$
DIM = DIM + 1 (Obtener siguiente dimensión)
HASTA que no haya más dimensiones
K = SUMA
Almacenar -K **K es la cte. de la fórmula
8.- REPETIR
identificador=pop PilaO
Almacenar BASE y tipo en el identificador
BASE = BASE + AUX
HASTA que PilaO quede vacía.

Para traducir la referencia a una variable dimensionada considerando que la declaración de la variable se efectuó bajo el método anterior, podríamos definir las siguientes acciones:

Acceso a VARIABLES DIMENSIONADAS



1.- PUSH PilaO (id)
2.- id = POP PilaO
Verificar que id sea una variable dimensionada
DIM = 1
PUSH PilaDimensionadas(id, DIM)
Obtener primer campo de descripción de id.
PUSH POper (marca de fondo falso)
3.- Generar cuádruplo:
Verifica tope-PilaO Li_{DIM} Ls_{DIM}
Nota: no se saca el top de la pila de operandos.
SI siguiente apuntador es diferente de nulo ENTONCES
aux = POP PilaO
T = temporal del avail
Generar *** aux m_{DIM} T**
PUSH PilaO (T)
SI DIM>1 ENTONCES
aux2 = POP PilaO
aux1 = POP PilaO
T = temporal del avail
Generar cuádruplo **+ aux1 aux2 T**
PUSH PilaO (T)
4.- DIM = DIM + 1
Actualizar DIM en PilaDimensionadas
Obtener siguiente campo de descripción
5.- aux1 = POP PilaO
T = temporal del avail
Generar cuádruplos:
\+ aux1 K T **K es la cte.
\+ T BASE T
PUSH PilaO (**(T)**) (*Para distinguirlo *)
POP POper (* eliminar marca *)
POP PilaDimensionadas

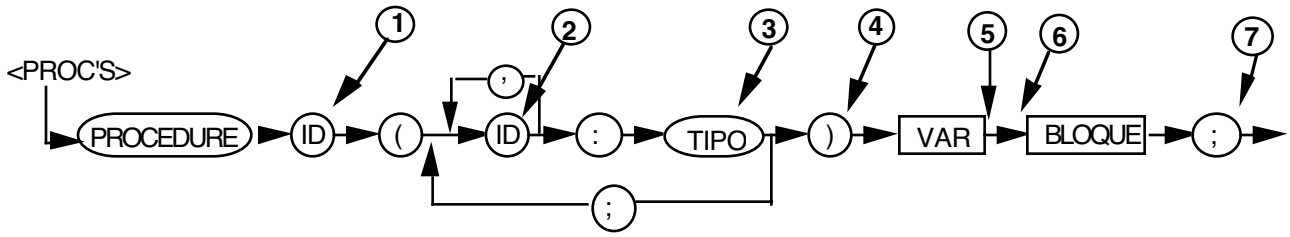
Nota : Se debe añadir la verificación semántica para reconocer si coincide la cantidad de dimensiones declarada, con la referida.

GENERACIÓN DE CÓDIGO PARA MÓDULOS

Hasta el momento se ha supuesto que los estatutos dentro del programa se presentan estrictamente en forma secuencial y dentro del programa principal. Sabemos que, en lenguajes como Pascal y C, el manejo de Procedimientos y Funciones es esencial para el desarrollo de "buenos" programas.
En buena parte, esto también pudiera ser utilizado cuando el lenguaje soporta la definición de clases y su consecuente llamada a los métodos de duchas clases.

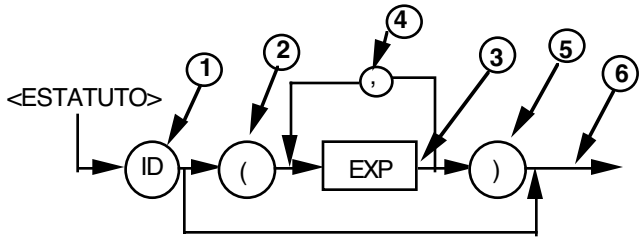
A continuación se definen las acciones de generación de código que se deben llevar a cabo para los módulos. Nuevamente se trabaja con una sintaxis similar a la del lenguaje Pascal, sin embargo esto puede ser extrapolado a cualquier tipo de sintaxis particular.

Acciones para la definición de un PROCEDIMIENTO



1.- Dar de alta al nombre del proc. en el Directorio del Procedimientos, verificar su semántica.
2.- Ligar cada parámetro a la tabla de parámetros del directorio de Proc's.
3.- Dar de alta el tipo de los parámetros.
4.- Dar de alta, en el Dir. de Proc's, el número de parámetros declarados.
5.- Dar de alta, en el Dir. de Proc's, el número de variables locales definidas.
6.- Dar de alta, en el Dir. de Proc's, el número de cuádruplo (CONT) en el que inicia el procedimiento.
7.- Liberar la tabla de variables locales del procedimiento.
Generar una acción de RETORNO.

Acciones para la llamada a un PROCEDIMIENTO



1.- Verificar que el procedimiento exista como tal en el Dir. de Proc's.
2.- Generar acción: ERA tamaño (expansión del registro de activación, según # de variables).
Inicializar contador de parámetros (k) en 1.
Apuntar al primer parámetro, dentro de la tabla de parámetros de ese procedimiento.
3.- Argumento = Pop de PilaOperandos, TipoArg = Pop de PilaTipos.
Verificar el tipo del argumento contra el del parámetro k.
Generar PARAMETRO, Argumento, Parámetro k *(*Se suponen Parámetros por Valor *)*
4.- K = K + 1, apuntar al siguiente parámetro.
5.- Verificar que el último parámetro apunte a Nulo. (para congruencia en # de parámetros).
6.- Generar GOSUB, nombre-proc, dir. de inicio .

Acciones para FUNCIONES y PARÁMETROS POR REFERENCIA.

Es responsabilidad del usuario extrapolar lo hecho para procedimientos, de tal forma que se pueda generar código para Funciones.

Además, es responsabilidad del usuario diseñar las acciones de código necesarias para soportar Parámetros por Referencia.

EJECUCIÓN DE ALGUNOS CÓDIGOS DE OPERACIÓN ESPECIALES PARA MÓDULOS.

ERA tamaño :
Salvar la base local actual (previa a la llamada). Actualizar la base local. Generar el espacio de trabajo para las variables locales y los parámetros del procedimiento.

GOSUB nombre-proc, dir. de inicio :
Meter la dirección de retorno en la pila de ejecución. Transferir el control de ejecución a la dirección de inicio del procedimiento.

RETORNO :
Actualizar base local (previa a la llamada). Destruir el registro de activación del proc. Recuperar la dirección de retorno y transferir el control de ejecución.

ADMINISTRACIÓN DE LA MEMORIA

ADMINISTRACIÓN DE LA MEMORIA AL MOMENTO DE GENERAR CÓDIGO INTERMEDIO.

a) Buffer finito.

Ocupar un buffer que se llene y si el programa requiere mas espacio marcar error "Programa demasiado largo" .

- desventaja: No es posible compilar programas muy largos.

b) Paginación.

Utilizar un buffer al que llamaremos página y si se nos termina lo vaciamos en el archivo de código, y de nuevo ocupamos el buffer desde el inicio.

- Usando esta estrategia es posible compilar programas de longitud infinita.
- Desventaja: Es posible que en un momento se ejecute una acción de rellenar un cuádruplo que podría encontrarse ya en el archivo, así que hay que accesar el cuádruplo correspondiente, modificarlo y volverlo a grabar, de esta manera la compilación se vuelve muy lenta, y también se volvería muy lenta la ejecución de este programa, de esto hablaremos posteriormente.

Para esta estrategia, el traductor debe crear un diccionario de páginas que indique en qué parte del archivo quedó grabada cada página.

c) Segmentación.

Dividir el programa en particiones lógicas a las que llamaremos segmentos. Cada partición debe asegurar que no tiene cuádruplos incompletos.

Se puede utilizar un buffer para almacenar el código de un segmento y al terminar de compilarse un segmento se vacía el buffer al archivo.

- Con esta estrategia es posible compilar programas de longitud infinita.
- Desventajas: si se graban solo los cuádruplos ocupados el proceso de escritura se puede volver muy lento, y si se graba el buffer completo se puede desperdiciar mucho espacio.

Si al compilar un segmento se termina el buffer se puede marcar el error "Procedimiento demasiado grande"

Para esta estrategia, el traductor debe crear un directorio de segmentos que indique en qué parte del archivo quedó grabado cada segmento.

d) Segmentación con paginación.

En esta estrategia el programa se divide en segmentos y el buffer que se utiliza se divide en páginas, de tal modo que al grabarse el código del segmento en el archivo se graba un número entero de páginas.

- El proceso de escritura en el archivo es más rápido que en la estrategia anterior, y también será más rápido el acceso al archivo en ejecución.
- Desventaja: Se puede desperdiciar una fracción de una página.

Para esta estrategia, el traductor debe crear un directorio de segmentos que indique en qué parte del archivo quedó grabado cada segmento, y cuántas páginas ocupa el segmento.

ADMINISTRACIÓN DE LA MEMORIA AL MOMENTO DE EJECUCIÓN

A continuación se mencionan algunas estrategias para cargar un archivo de código intermedio en memoria para su ejecución.

Cada paréntesis hace referencia a las estrategias vistas en el tema de "Administración de la memoria al momento de generar código intermedio".

a) Buffer finito.

Se carga el archivo completo de código intermedio en un buffer de igual tamaño que el que se utilizó en compilación y se ejecuta el código.

b) Paginación.

Se define un buffer cuyo tamaño sea igual a una o mas páginas. A este buffer le llamaremos área de overlay.
Para iniciar la ejecución del código se deben cargar al menos las páginas que contengan el programa principal (a su inicio si es muy largo).
Al grabar el código se debe dejar grabada la información del número de página donde inicia el programa principal.
Cada vez que ocurra un salto a un cuádruplo que no esta presente en memoria (en el área de overlay) se deberá utilizar alguna fórmula para calcular en que página se encuentra dicho cuádruplo.
En ejecución se deberá crear un directorio para llevar el registro de las páginas que se encuentran cargadas en el área de overlay.

Si al cargar una mágina el area de overlay está llena se debería seguir alguna estrategia de desocupación (como las que vieron en sistemas operativos). La desventaja de este método es que puede ocurrir un gran tráfico de páginas que pueder volver muy lenta la ejecución.

c) Segmentación.

Si el programa se dividió en segmentos, el momento de compilación se debió crear un directorio de segmentos que indique en que lugar del archivo quedó grabado cada segmento, así como su tamaño.
Para cargar el archivo de código para su ejecución existen dos alternativas.

i) que el usuario decida qué procedimientos desea que se encuentren residiendo en todo momento en memoria principal y cuales trafiquen en una area overlay.
ii) que el programa decida qué procedimientos trafiquen por el área de overlay.

* Si se utiliza la alternativa i se debe tener un buffer para cargar todos los procedimientos no segmentados (como ocurre en Pascal) (El usuario debió de haber distinguido con alguna marca, como la palabra segment, a los procedimientos resdentes de los no residentes) Si no caben en el buffer todos los segmentos residentes se deberá marcar un error (como stack overflow) y el usuario deberá dejar no residentes a un mayor número de segmentos.
Si sí caben en el buffer todos los segmentos residentes, entonces se deberá utilizar un área de overlay (o pila) para ir cargando a los segmentos no residentes cada vez que sean llamados, y deberán de sacarse al terminar su ejecución. Si en algún momento se terminara el area de overlay se marcaría el error de falta de memoria (stack overflow).

* Si se utiliza la alternativa ii se deberán cargar en un area de overlay todos los segmentos que sea posible, incluyendo por supuesto al segmento que contiene al programa principal.

Se deberá seguir alguna estrategia de desocupación para cuando haya que sacar algún segmento para cargar otro que requiere ejecución. Se debe llevar control en ejecución de los segmentos cargados en overlay.
El problema que puede presentar esta estrategia C es que los segmentos quedan de tamaño irregular y puede quedar lento el acceso al archivo, por tener que cargarse cuádruplo por cuádruplo.

d) Segmentación con paginación.

Esta estretegia es similar a la anterior las diferencias son:

- En el directorio que se crea en compilación se debe tener información sobre el número de páginas que contiene un segmento, y la dirección de inciso del segmento dentro del archivo, queda relativa al número de página donde comienza.
- El acceso al archivo queda mucho más rápido por traficar con páginas de 6 bloques completos, de esta manera es posible utilizar instrucciones tales como Blockread que son más rápidas que las instrucciones como el get.
- Se puede evitar desperdicio del área de overlay por tener fragmentos de página vacíos teniendo la información.

BIBLIOGRAFIA

- Compilers: Principles, Techniques and Tools
  Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman
  Addison Wesley, 1986

- The Theory and Practice of Compiler Writing
  Jean-Paul Tremblay, Paul G. Sorenson
  McGraw-Hill

- Compiler Construction Theory and Practice
  William A. Barret, Rodney M. Bates,
  David A. Gustafson, John D. Couch

- Crafting a Compiler
  Charles N. Fischer, Richard J. Le Blanc, Jr.

- Introduction to Computer Theory
  Daniel I. A. Cohen
  John Wiley & Sons, Inc.

- The Theory of Parsing, Translation and Compiling; vol 1: Parsing.
  Alfred Aho, Jeffrey Ullman
  Prentice-Hall , 1972.

- Formal Languages and Their Relation to Automata
  John Hopcroft, Jeffrey Ullman
  Addison Wesley , 1969

- Currents in the Theory of Computation
  Alfred Aho
  Prentice-Hall , 1973

- Theory of Finite Automata
  John Carroll , Darrell Long
  Prentice-Hall , 1989

- Compiler Construction for Digital Computers
  David Gries
  Wiley International , 1971

- Syntax of Programming Languages
  Roland Backhouse
  Prentice-Hall , 1979

- Machines, Languages and Computation
  Peter Denning, Jack Dennis, Joseph Qualitz
  Prentice-Hall , 1978

- Compiler Design Theory
  P. Lewis, D. Rosenkrantz , R. Stearns
  Addison Wesley , 1976